Model-Based Design Toolbox S32K3 Series

Release Notes

Automatic Code Generation for the S32K3 Family of MCUs
Version 1.6.0

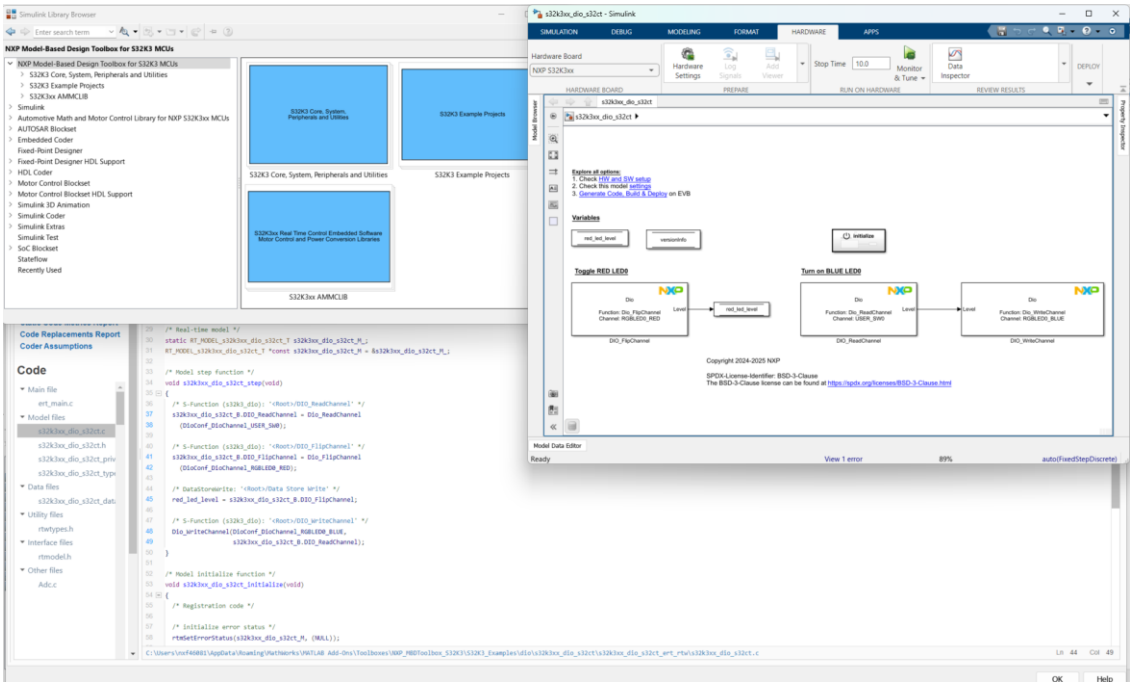
Target Based Automatic Code Generation Tools
For MATLAB™/Simulink™/Stateflow™ Models working with Simulink Coder ™ and Embedded Coder®

Summary

1 Main Features

The NXP's Model-Based Design Toolbox for S32K3 Series version 1.6.0 is designed to support S32K310, S32K311, S32K312, S32K314, S32K322, S32K324, S32K328, S32K338, S32K341, S32K342, S32K344, S32K348, S32K358, S32K364, S32K366, S32K374, S32K376, S32K388, S32K394 and S32K396 MCUs into MATLAB/Simulink environment, allowing users to:

- **Design** applications using Model-Based Design methodologies;
- **Simulate** and **Test** Simulink models for S32K3 MCUs before deploying the models to the hardware targets;
- **Configure** the MCU peripherals from Simulink Blocks via external configuration tools: NXP S32 Configuration Tools or EB tresos;
- **Generate** the application code automatically without any needs for hand coding C/ASM;
- **Deployment** of the application directly from MATLAB/Simulink to the NXP evaluation boards;



The main features and functionalities supported by the release v.1.6.0 are:

- Support for S32K310, S32K311, S32K312, S32K314, S32K322, S32K324, S32K328, S32K338, S32K341, S32K342, S32K344, S32K348, S32K358, S32K364, S32K366, S32K374, S32K376, S32K388, S32K394 and S32K396 MCUs
- Integrated with S32Configuration Tool (v2021.R1.7 Update 8) and EB tresos (v29.0.0)
- Integrates the Real-Time Drivers (v5.0.0)
- Integreates eTPU Software Drivers v2.0.0 CD01 to enable eTPU co-processor access from Simulink models
- Motor control examples (1-shunt, 2-shunt PMSM, BLDC, PMSM using the eTPU co-processor)
- Compatible with MATLAB releases R2021a, R2021b, R2022a, R2022b, R2023a, R2023b, R2024a, and R2024b
- Fully integrated with Simulink Toolchain

- Includes extensive Simulink Library Blocks for S32K310, S32K311, S32K312, S32K314, S32K322, S32K324, S32K328, S32K338, S32K341, S32K342, S32K344, S32K348, S32K358, S32K364, S32K366, S32K374, S32K376, S32K388, S32K394 and S32K396 MCUs, providing dedicated NXP blocks for peripheral components (e.g.: ADC, CAN, eTPU (blocks for Etpu, MotorControl, and Rdc_Checker components), DIO, FEE, GPT, I2C, ICU, LIN, MEM, MCL, PWM, SPI, UART)
- Supports FreeMASTER drivers v1.3.0
- Supports AMMCLib v1.1.39
- Supports AUTOSAR blockset (SW-C deployment)
- Supports download via JTAG and OpenSDA
- Includes an Example library with approximatively 140 examples that cover a wide range of topics like:
    - I/O control
    - Timers and scheduling
    - Communication
    - Motor control (BLDC and PMSM)
    - Memory handling
    - Software-in-Loop, Processor-in-Loop, External mode, Profiling

For more details about each of the topics highlighted above please refer to the following chapters.

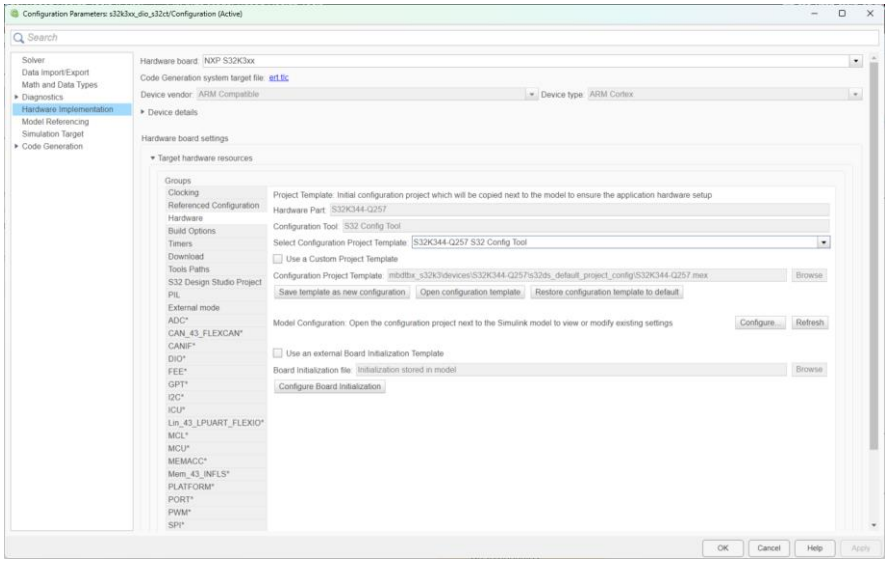
2 S32K3 MCU Support

2.1 Packages & Derivatives

The NXP's Model-Based Design Toolbox for S32K3 Series version 1.6.0 supports:

- S32K3 MCU Packages:
    - S32K310-100HDQFP
    - S32K311-100HDQFP
    - S32K312-172HDQFP
    - S32K314-172HDQFP
    - S32K322-172HDQFP
    - S32K324-172HDQFP
    - S32K328-172HDQFP
    - S32K338-172HDQFP
    - S32K341-172HDQFP
    - S32K342-172HDQFP
    - S32K344-257MAPBGA
    - S32K344-172HDQFP
    - S32K348-172HDQFP
    - S32K358-172HDQFP
    - S32K364-289MAPBGA
    - S32K366-289MAPBGA
    - S32K374-289MAPBGA
    - S32K376-289MAPBGA
    - S32K388-289MAPBGA
    - S32K394-289MAPBGA
    - S32K396-289MAPBGA

The configurations can be easily changed for each Simulink model by expanding the **Target hardware resources** tab, and accessing the **Hardware** group. Step by step instructions for customizing a model for a certain MCU or hardware board will be depicted in the following chapters of this document.

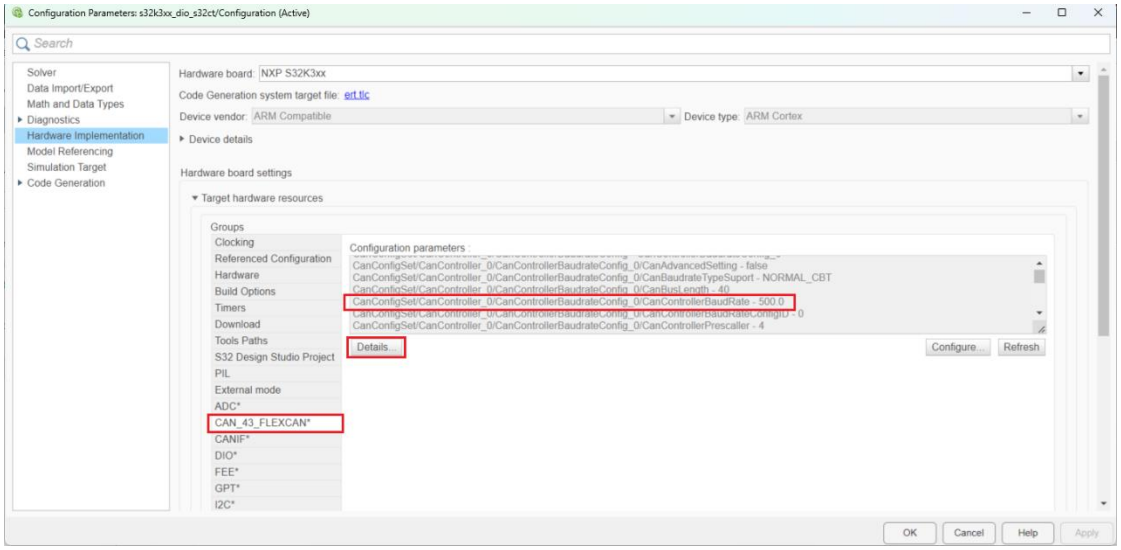


2.2 Peripherals and functions

The NXP's Model-Based Design Toolbox for S32K3 Series version 1.6.0 supports the following peripheral components and functions:

- ADC
- CAN
- eTPU (Etpu, MotorControl, and Rdc_Checker components)
- DIO
- FEE
- GPT
- I2C
- ICU
- LIN
- MEM
- MCL
- PWM
- SPI
- UART
- Memory read/write
- Register read/write
- Profiler

The default configuration supported by the toolbox for each peripheral is available inside the **Target hardware resources** panels:



From this panel, the user can access the hardware documentation (Reference Manual) using the **Details**… button. To update the model configuration, the user can click on the **Configure…** button to open the NXP S32 Configuration Tool / EB tresos. The selected external configuration tool will be opened and the user will have access to the entire parameter list for pins / clock / peripherals.

The NXP's Model-Based Design Toolbox for S32K3 Series version 1.6.0 has been tested using the official NXP Evaluation Boards for S32K3:

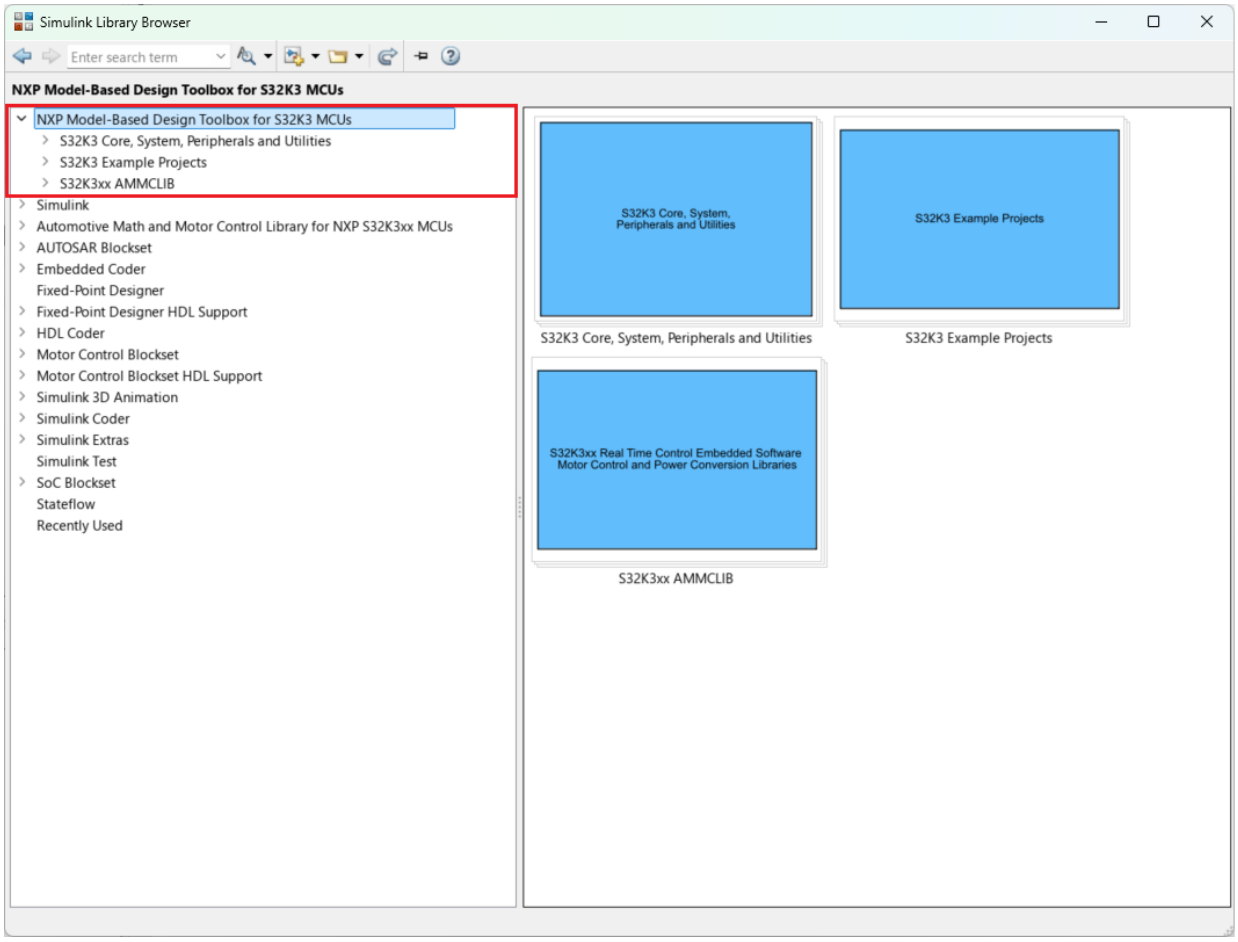
- S32K31XEVB-Q100
- S32K312EVB-Q172
- XS32K3X2CVB-Q172
- XS32K3X4EVB-Q257
- XS32K3XXEVB-Q172
- MR-CANHUBK344
- S32K3X4EVB-T172
- S32K344-WB
- XS32K3X8CVB-Q172
- S32K388EVB-Q289
- XS32K396-BGA-DC
- XS32K396-BGA-DC1

3 Model-Based Design Toolbox Features

The NXP's Model-Based Design Toolbox for S32K3 Series version 1.6.0 is delivered with complete S32K3 MCUs Simulink Block Library as shown below.

There are three main categories:

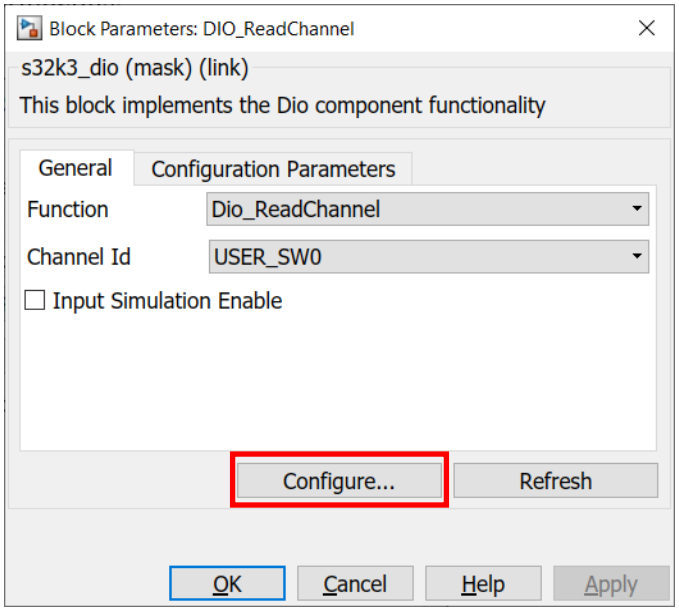
- S32K3xx AMMCLib
- S32K3 Core, System, Peripherals and Utilities
- S32K3 Example Projects



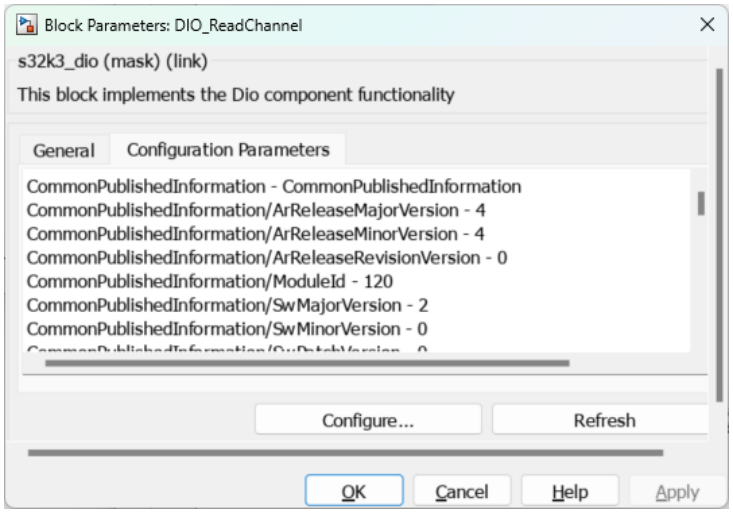
3.1 S32K3 Simulink Library Blocks

All Simulink blocks supported by the NXP Toolbox are designed to offer the best out-of-the-box experience, providing a basic peripheral configuration that covers most of the hardware capabilities. Most of the Simulink Blocks contains just two tabs:

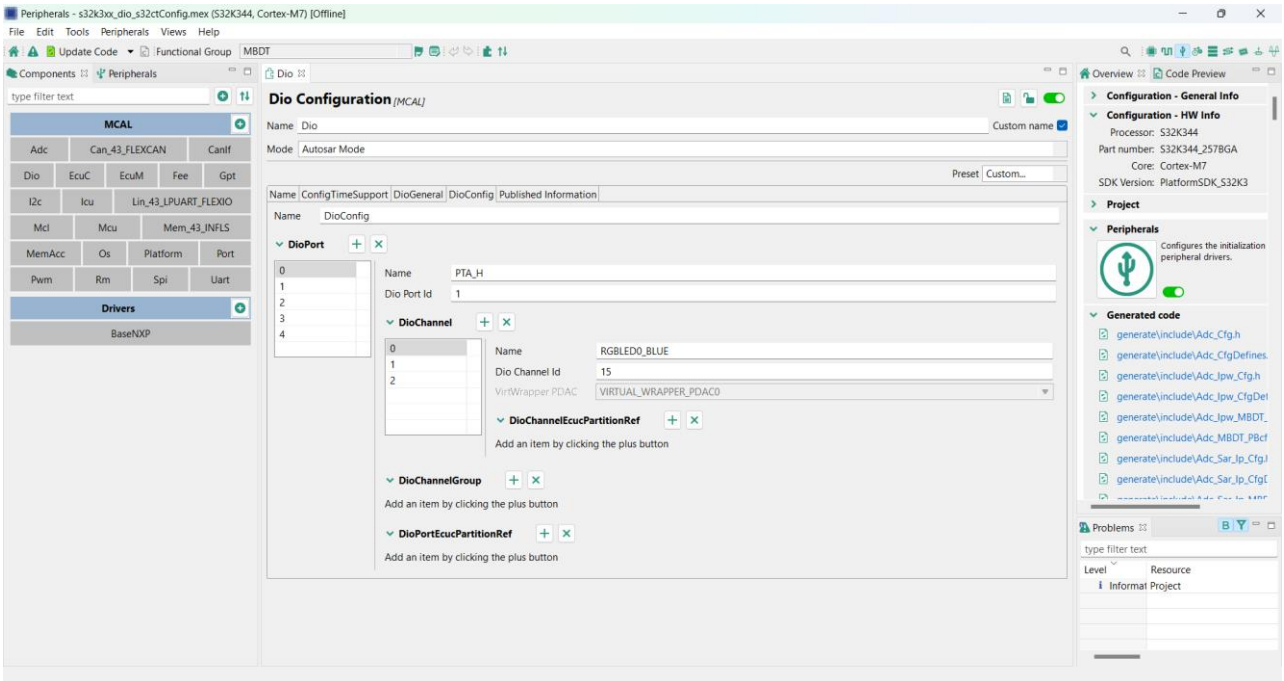
1. **General** Tab allows the user to select what functionality he wants the block to enable (usually this will be a list of MCAL functions). Besides that, the user can edit some parameters (this will be different, depending on the function selection). All peripheral components come with a working configuration (the toolbox default setup). These settings have been chosen to work out-of-the-box for the supported EVBs. The toolbox can be used as-is if no configuration parameters have to be changed. This is the BASIC mode of operation for the toolbox. In the ADVANCED mode, the user can click on the **Configure…** button, which will open an external configuration tool (NXP S32Configuration Tool / EB tresos). There, the user has full access to all the parameters needed to set up the pins/clock/peripherals.

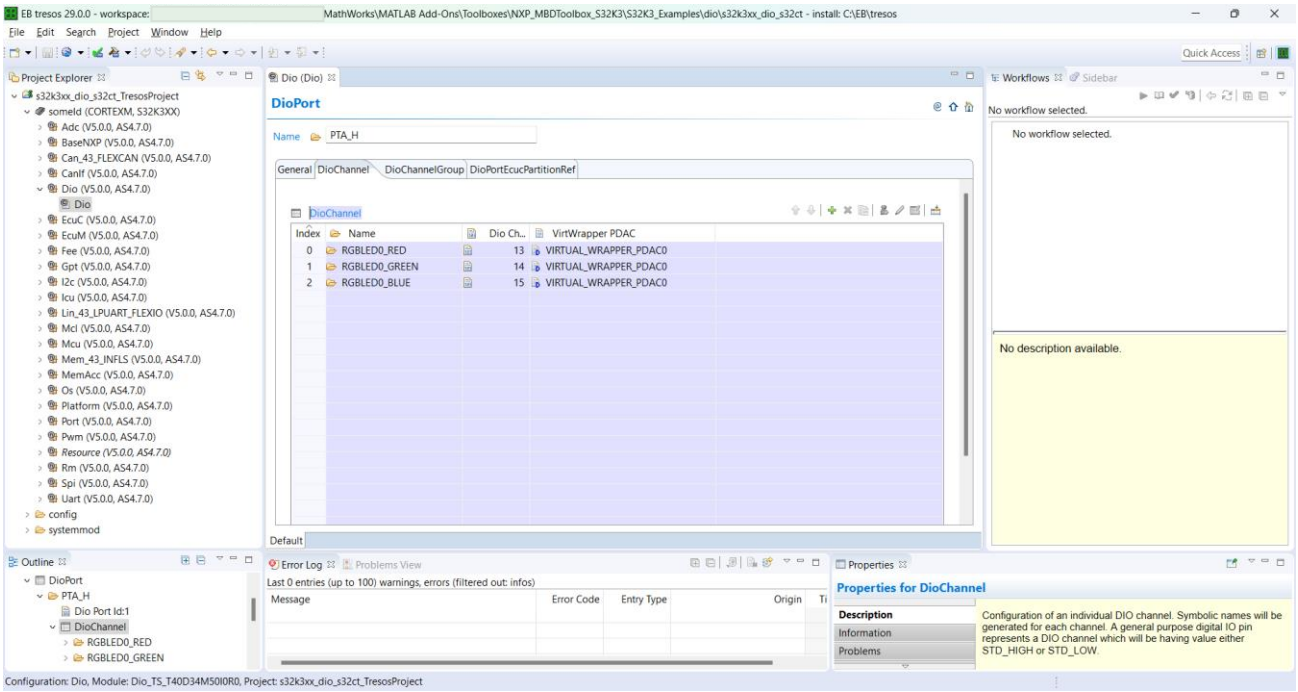


2. **Parameters** Tab which contains the detailed configuration available. This information is shown for verification purposes.



From any of these blocks, by clicking on the **Configure…** button, the users can open the external configuration tool of choice (NXP S32Configuration Tools / EB tresos) to alter the default configuration used by the Simulink model. The **Refresh** button will bring the new made settings inside the Simulink environment, so that they can be accessed from the library blocks.



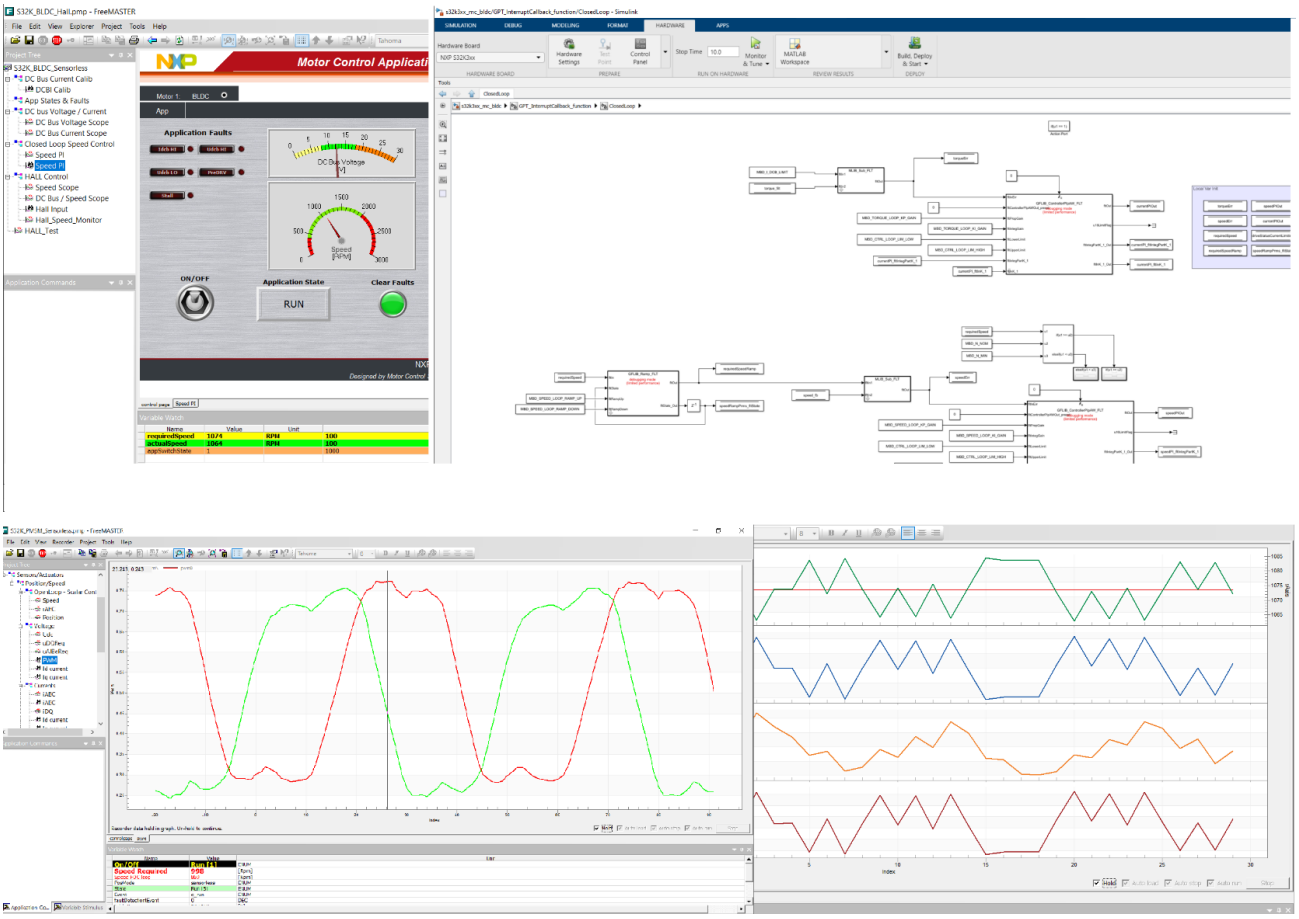


Using the external tools, the users can perform ADVANCED configurations and then use these new options in Simulink models. The validation of the configuration and peripheral code generation is done outside of Simulink.

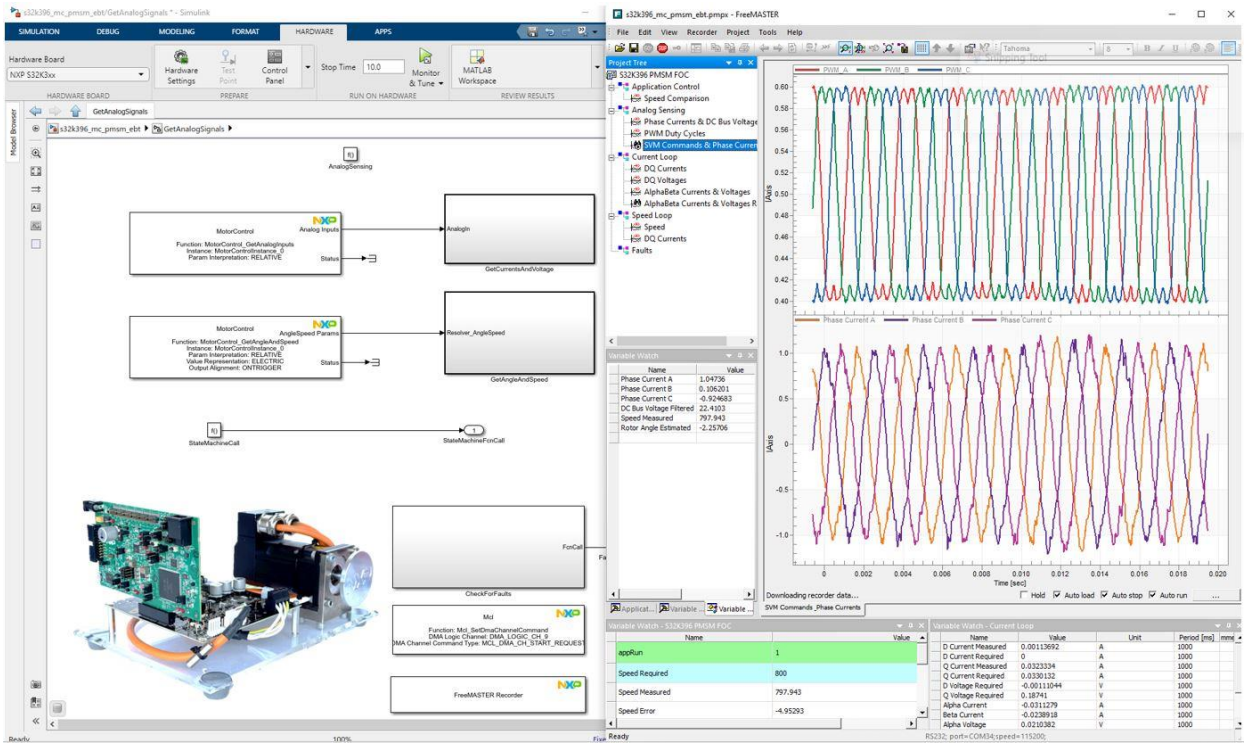
3.2 Motor control examples

The toolbox provides examples for both **1-shunt, 2-shunt PMSM** and **BLDC** motor control applications, supporting both **S32 Configuration Tools** and **EB tresos**. Each of them has a detailed description of the hardware setup and an associated FreeMASTER project which can be used for control and data visualization.

The toolbox also provides an example demonstrating the integration of the **Motor Control Blockset** (a MathWorks toolbox which provides reference examples and blocks for developing and implementing motor control algorithms) integration in developing such applications.

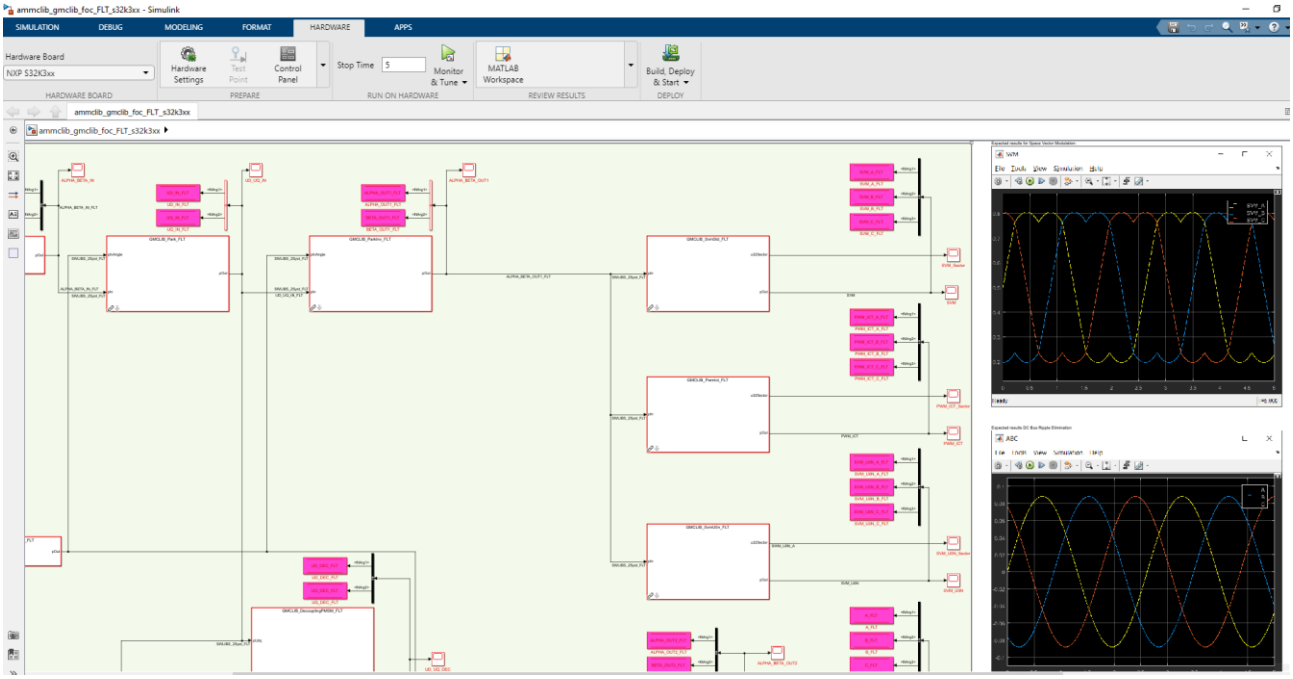


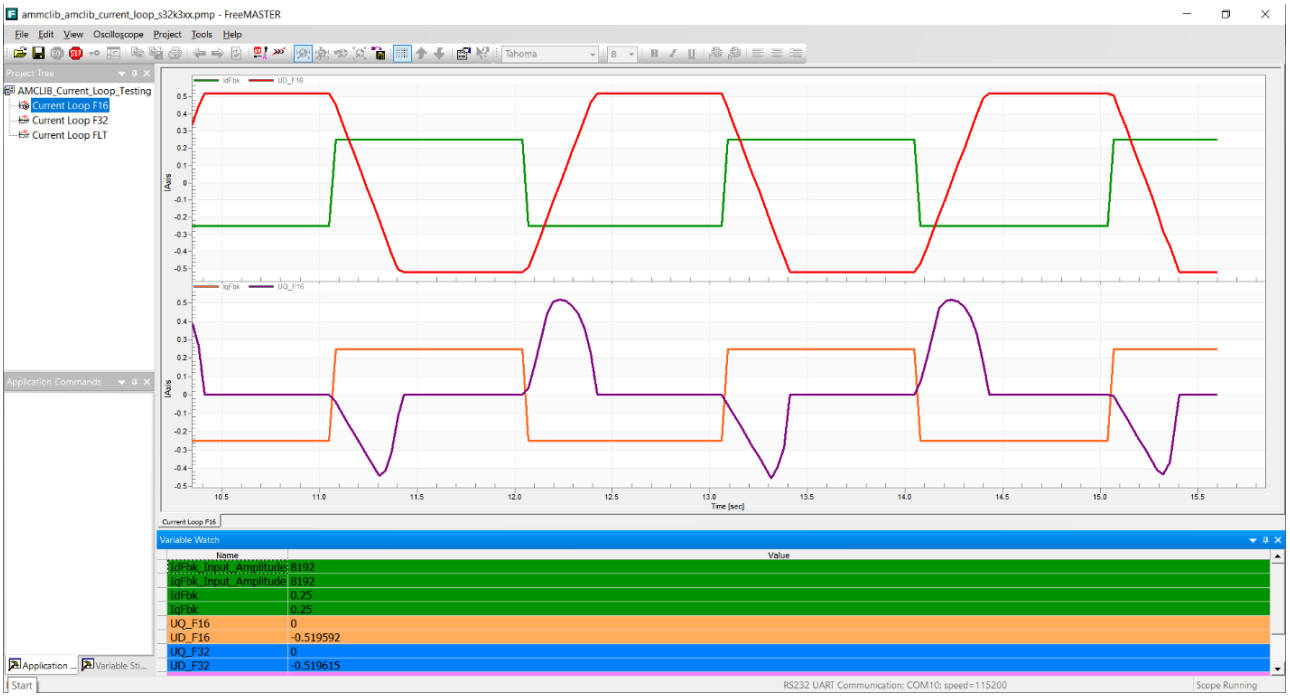
Moreover, the toolbox delivers a PMSM application, where the FOC algorithm runs on the main CPU of the S32K396 MCU, while the analog sensing, software resolver, and PWM signals generation are offloaded to the eTPU co-processor. For implementing this kind of motor control application, demonstrating how the eTPU co-processor of the S32K39x/S32K36x derivatives can be accessed from Simulink models, the toolboxes delivers Simulink blocks that generate code on top of eTPU drivers APIs, and an EB tresos configuration project for configuring the motor control components.



3.3 S32K3xx AMMCLib integration

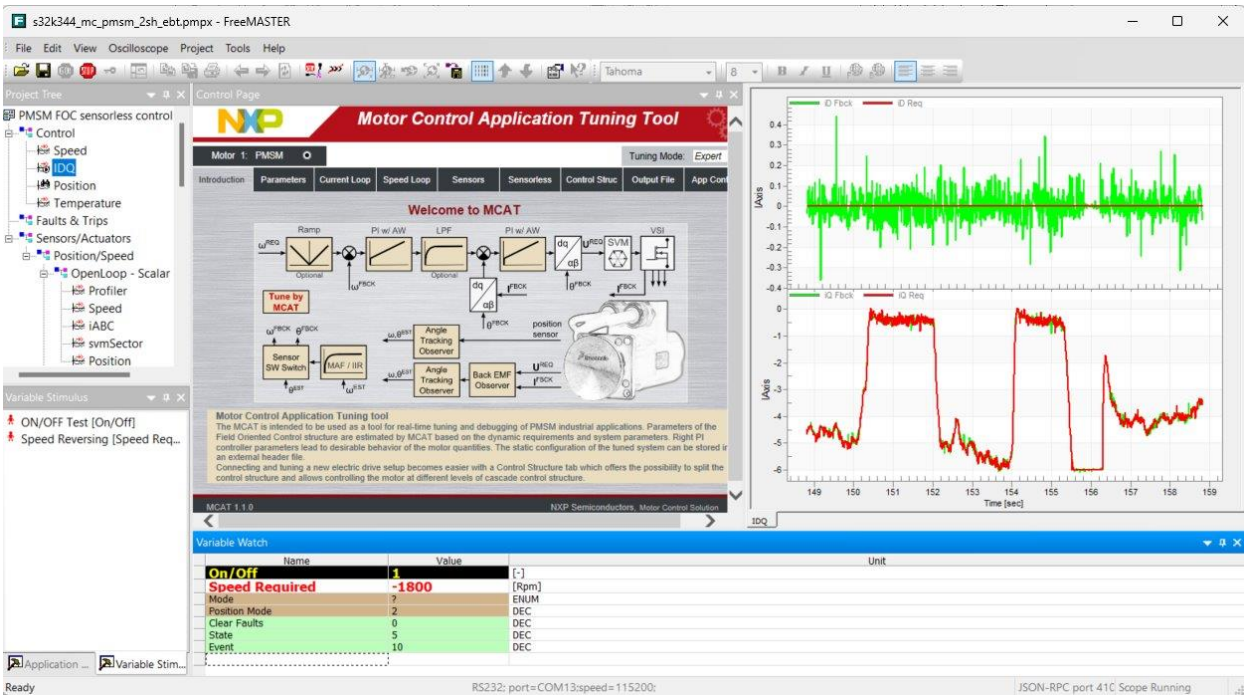
This toolbox also ships the AMMCLib v1.1.39 blocks. These blocks are essential for building applications with high-performance arithmetic, trigonometric, digital signal processing and math functions.





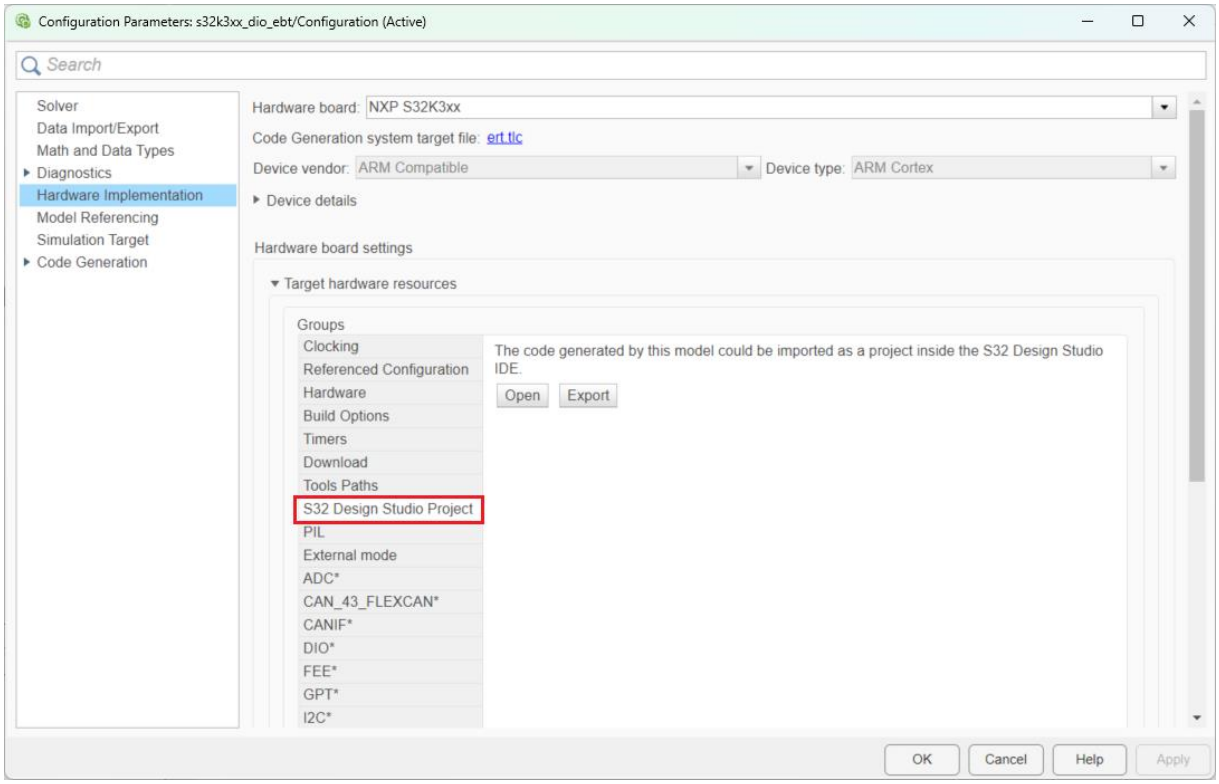
3.4 S32K3 FreeMASTER integration

This toolbox has support for FreeMASTER drivers version 1.3.0. FreeMASTER represents an user-friendly real-time debug monitor and data visualization tool that enables runtime configuration and tuning of embedded software applications via **UART** and **CAN**. With this support, users can build models and monitor various variables, display them on multiple scopes and even add widgets (gauges, sliders, etc).

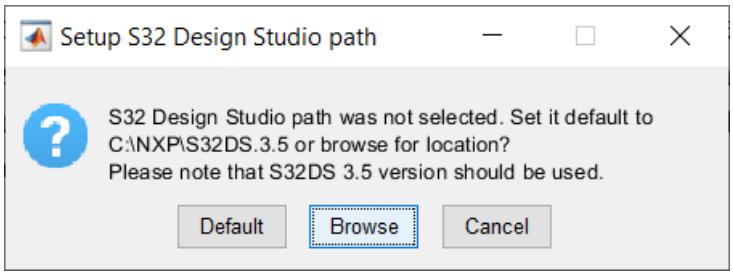


3.5 S32 Design Studio Integration

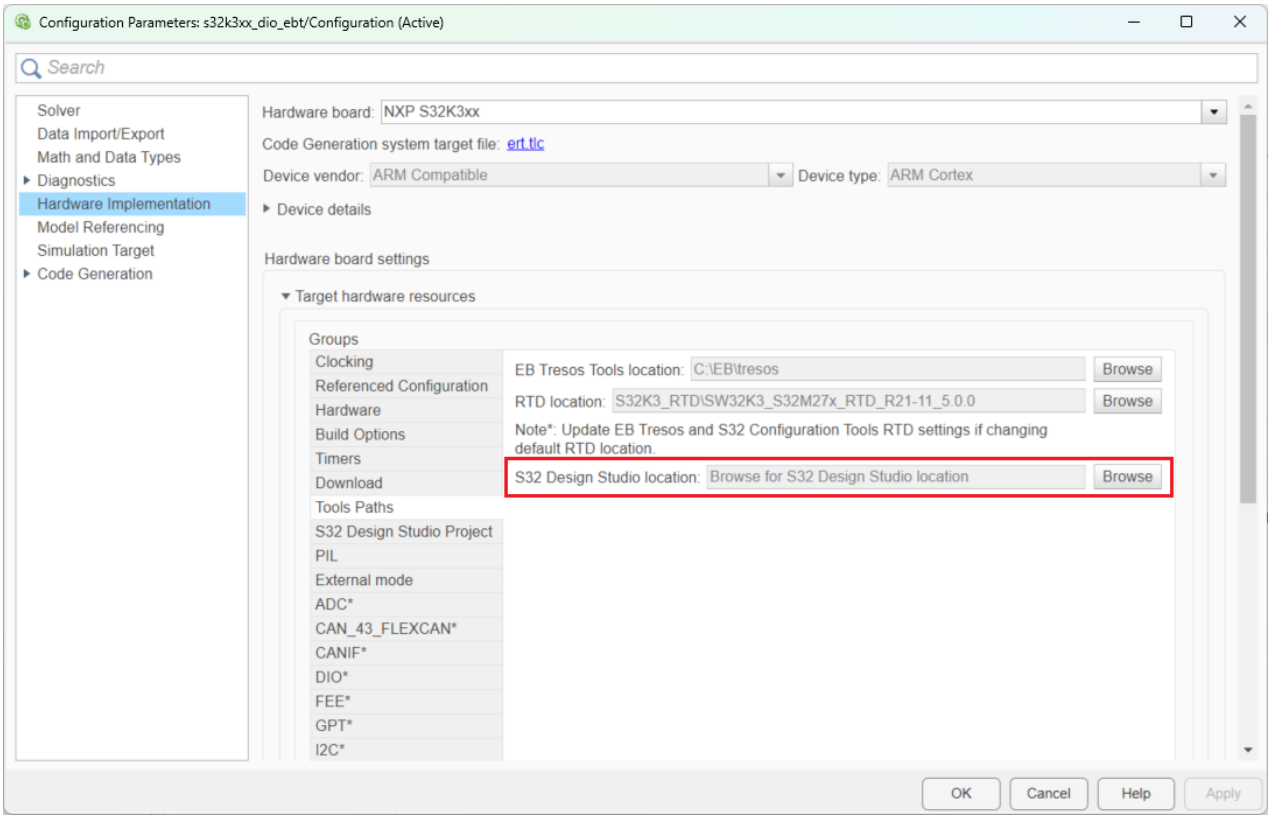
The toolbox allows the users to export the code generated from Simulink models and import it into the S32 Design Studio IDE. This functionality can be useful when the model needs to be integrated in an already existing project, or for debug purposes. After a model is built and the code generation process completes successfully, the <modelName>**_Config** folder existing next to the model is already in an IDE compatible format and can be opened inside S32 Design Studio, by pressing the **Open** button from the **S32 Design Studio Project** tab. The **Export** button allows users to save the IDE compatible project in the local file system.



When pressing the **Open** button, the users will be prompted with the following dialog, asking for the path of the S32 Design Studio installation:

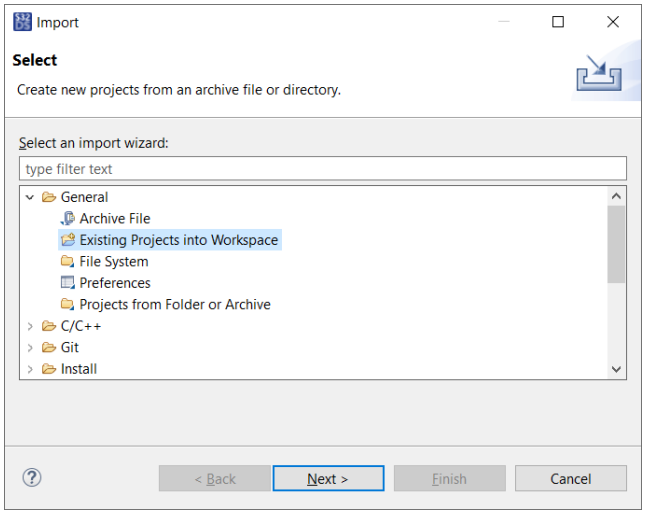


If the path selection is desired to be performed at a later date, or changed during the toolbox usage, the **Browse** button of the **S32 Design Studio location** widget, under the **Tools Paths** group, should be pressed to allow a new selection.

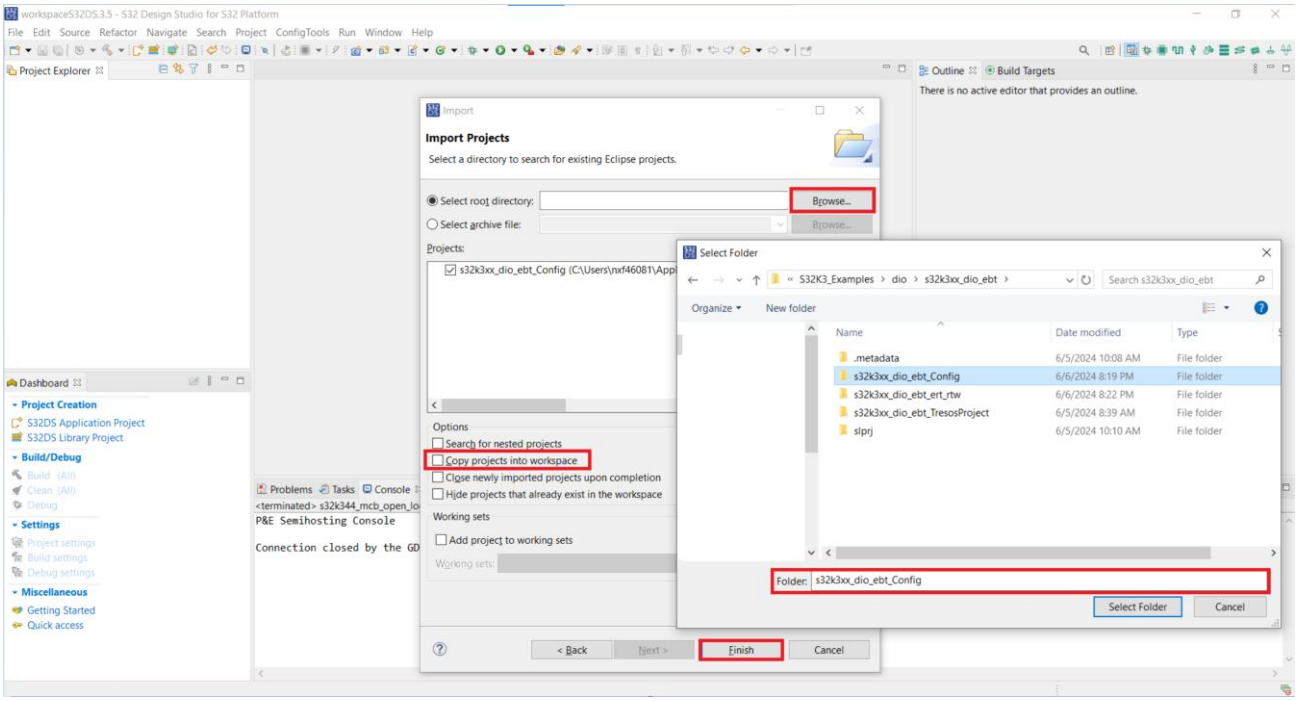


For a manual Import of the project inside the S32 Design Studio IDE, the following steps should be followed:

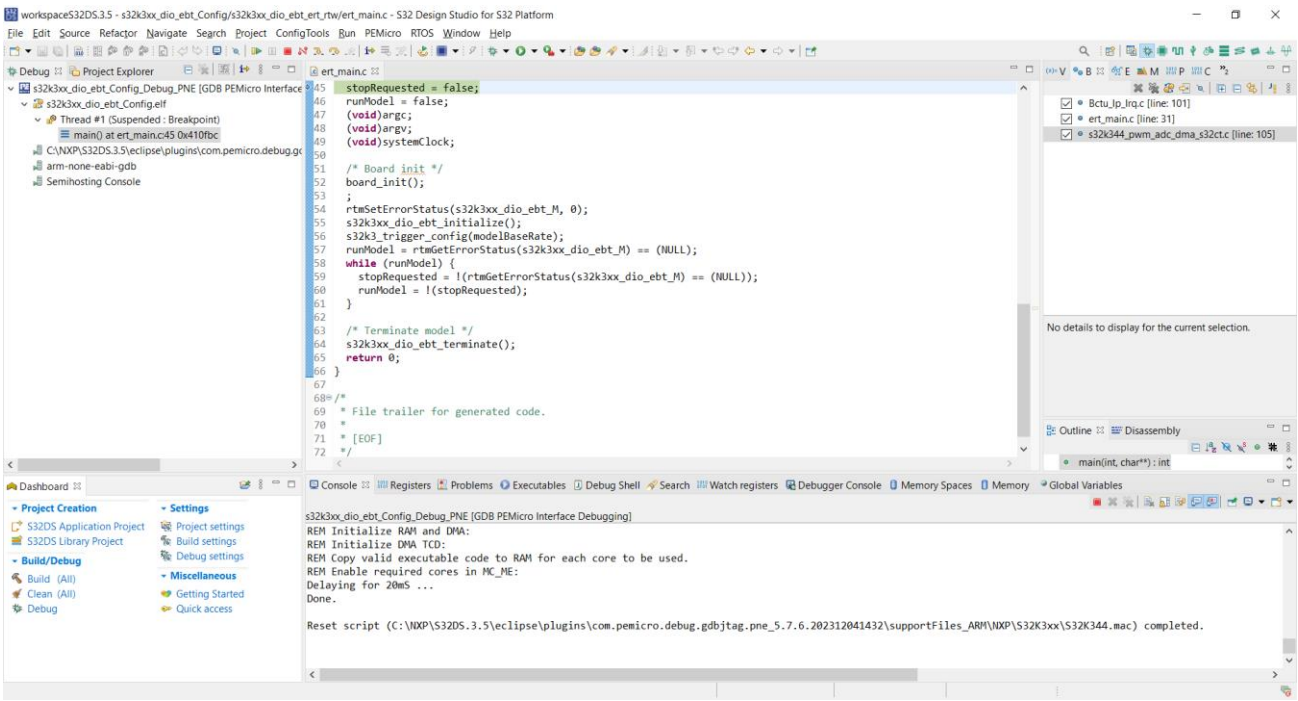
1. Inside the IDE, select **File** -> **Import** -> **Existing Projects into Workspace**



2. Browse for the **<modelName>_Config** folder inside the **Select root directoy** tab and before clicking **Finish**, make sure that the **Copy projects into workspace** checkbox is **DISABLED**. If the project will be copied into the S32 Design Studio workspace, the build process will fail.



3. The project can be built and debugged inside the **S32 Design Studio** environment



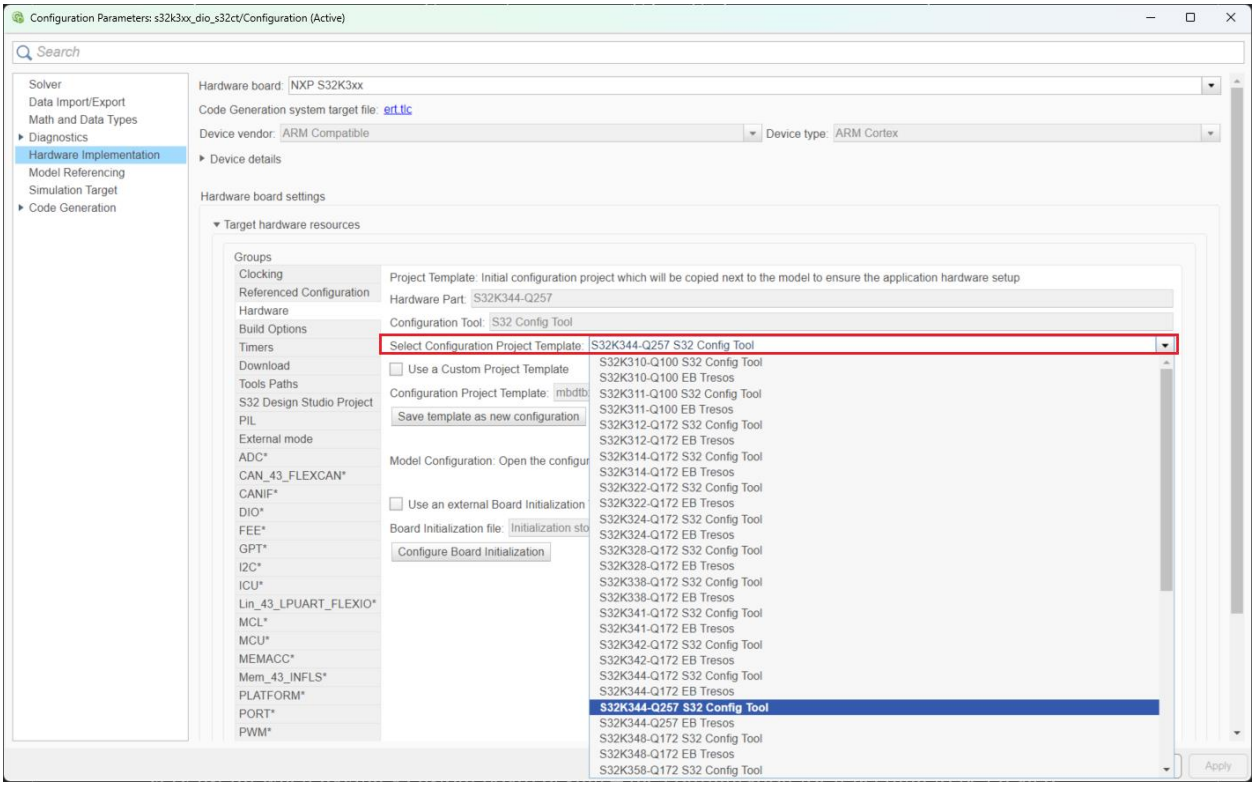
3.6 Support for Custom Default Project

Inside the **Hardware** group, there are two sections. The **Project Template** targets the initial configuration project which will be copied next to the model if no other configuration is found there. This will ensure the peripherals, pins, and clocks settings for the application being developed. Once a model is created, this configuration project will be copied next to the Simulink model and renamed accordingly. If the configuration project placed next to the model is deleted, the toolbox framework will copy it again, assuming this template as the starting point for enabling the hardware setup. Several templates are provided by the toolbox, covering an initial enablement for all the supported processors, and in some cases, different cores of the processor.

The **Model Configuration** section refers to the configuration project placed next to the Simulink model. The buttons **Configure…** and **Refresh** allow the access to the project, for modifying it as required, and for bringing back any modifications, from the external configuration tool, inside the Simulink environment.

Having established the difference, in terms of concept, between a Configuration Project Template and a Model Configuration, the following paragraphs will describe the possible settings, enabled by the toolbox framework, for manipulating the Configuration Project Template.

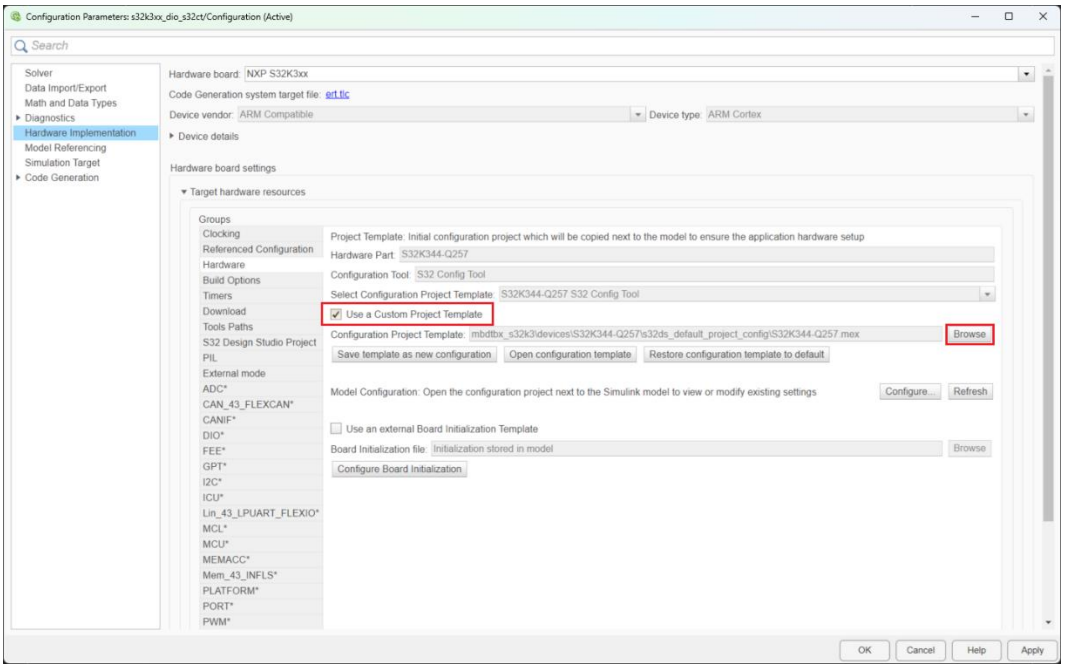
The **Select Configuration Project Template** dropdown widget will allow the selection of one of the default projects the toolbox provides, for both supported configuration tools (S32 Configuration Tools and EB tresos). Based on the selected configuration project, the **Core**, **Hardware Part** and the **Configuration Tool** parameters will be updated automatically.



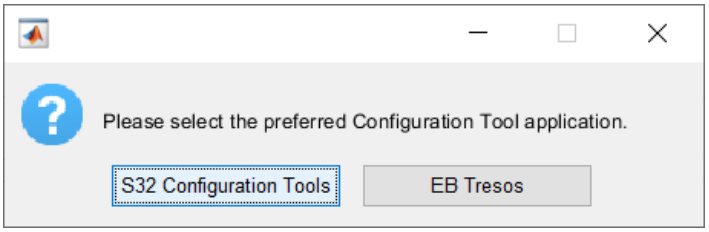
The toolbox provides support for users to create their own custom default project. This could be very useful when having a custom board design – the configuration for it needing to be created only once. After the configuration is saved as a custom default project, it can be used for every model that is being developed.

The toolbox already provides custom configuration projects for addressing specific hardware designs such as MR-CANHUBK344, S32K344-WB, and XS32K396-BGA-DC1. The default configuration projects created for the processors present on these boards, were already designed and validated on other hardware setups, hence, custom projects were created to enable the differences in terms of configuation for the peripherals, pins, and clocks on these boards.

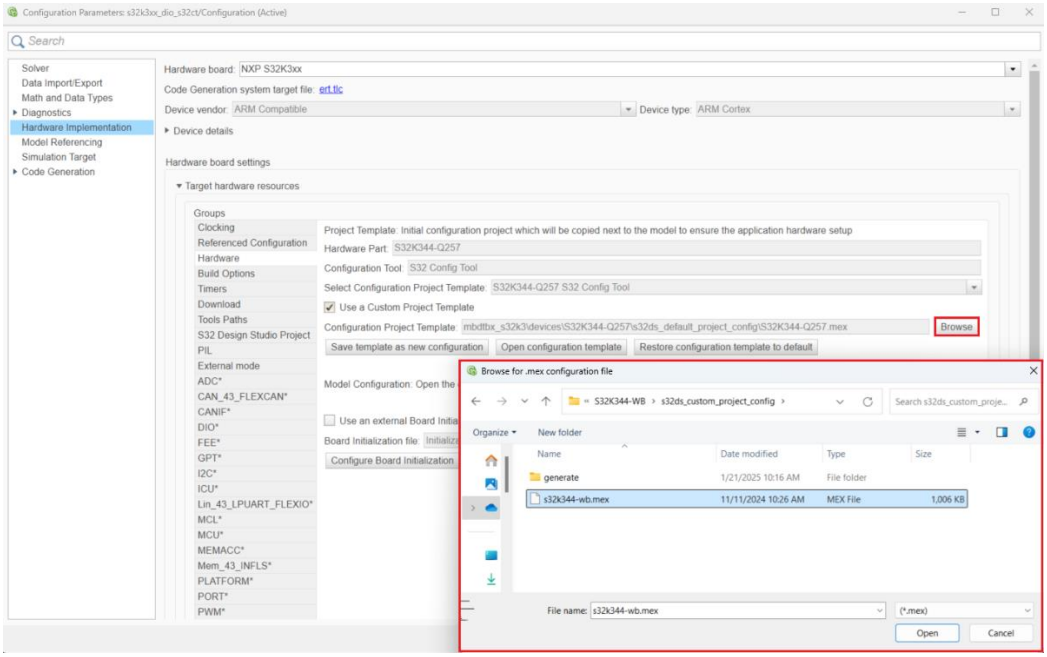
Hence, if a project different than the ones provided by the toolbox should be used, the **Use a Custom Project Template** checkbox should be enabled. This will allow the users to **Browse** for a configuration project that they would like to use in conjunction with the developed Simulink model.



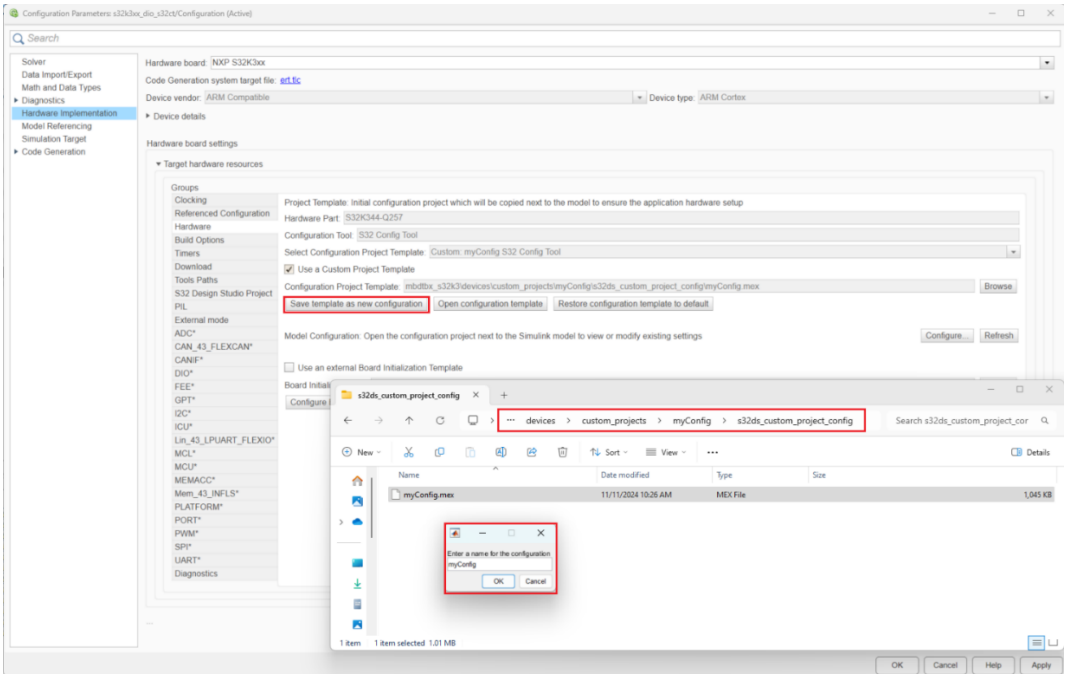
The **Browse** button will firstly provide a prompt asking for the selection of the Configuration Tool in which the Custom Project is designed.



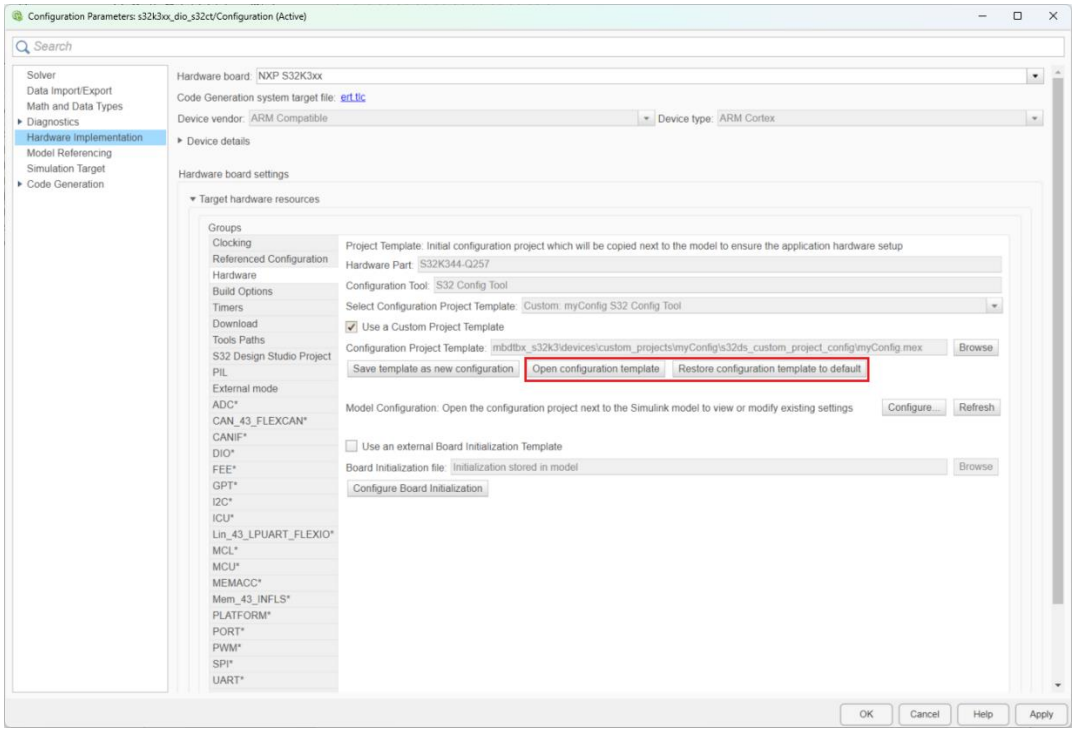
After the configuration tool is selected, the Windows Explorer will open, allowing the selection of a different project to be used as the default one for the model. Users can choose one of the projects delivered by the toolbox, or a different configuration they might have on their PCs.



The **Save template as new configuration** button allows saving the new selected template next to the already existing projects provided by the toolbox, under a name at the user's choice. The project will be stored inside the **custom_projects** folder. This newly saved configuration can be used for other models as well by selecting it using the previously described **Browse** functionality.



The **Open configuration template** button allows opening the **Configuration Project Template** in the selected Configuration Tool, where it can be furtherly modified and saved. When pressing the **Open configuration template** button, a backup copy of the **Configuration Project Template** will be created, such that users could **Restore configuration template to default** at any point during the development process.

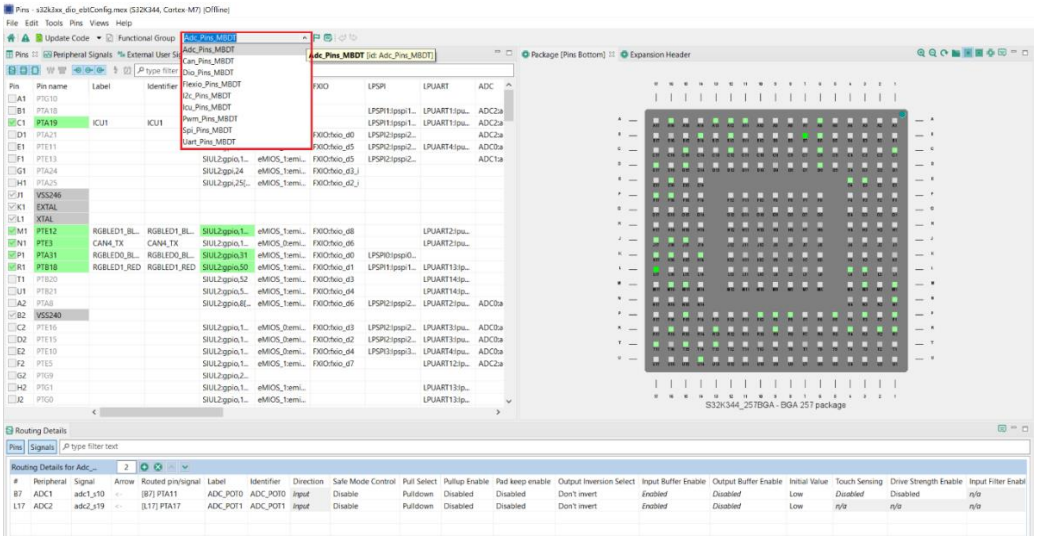


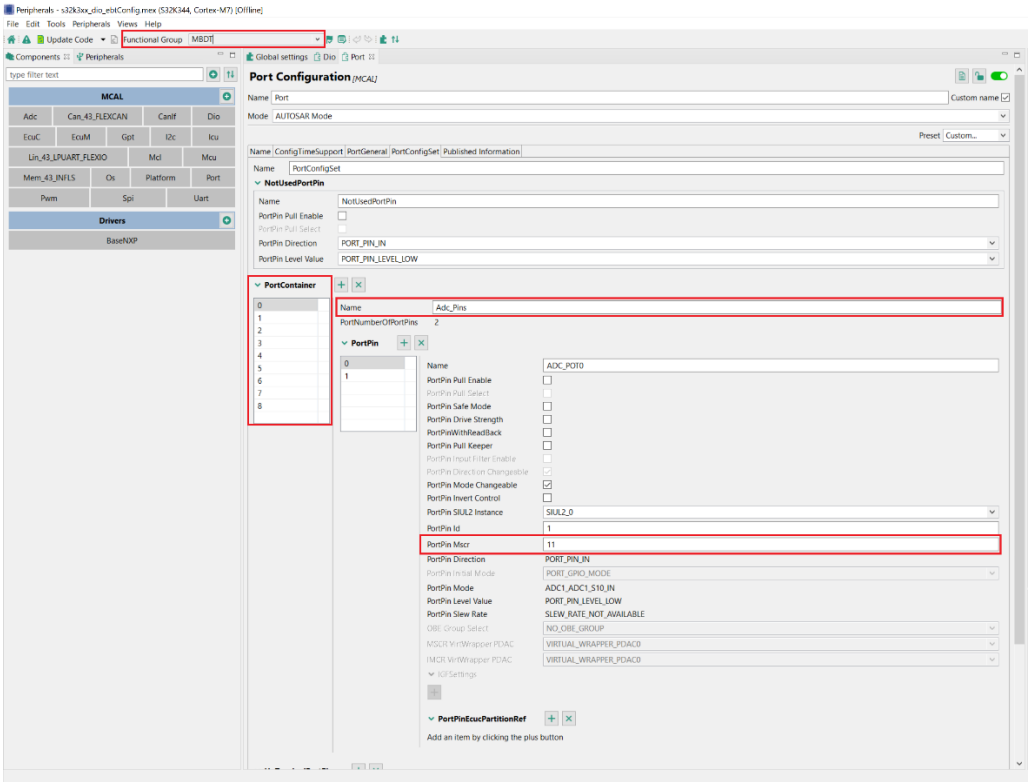
If using a custom project which **was not developed** starting from the default ones already delivered inside the toolbox, please note that the following configurations are **mandatory** to ensure a correct integration of the project with the framework of building the S32K3 Simulink models inside MBDT.

The list below refers to an S32 Configuration Tools .mex project. Please note that for EB tresos some of the items do not apply (e.g there are no Pins and Clocks Tools inside EB tresos). However, the below information provides an insight on the requirements, in terms of framework functionality, that the toolbox imposes on the configuration project associated to a model.

1. The toolbox offers support for configuring the on-board pins using the **Port** component, from the *Peripherals Tool*, together with the *Pins Tool*. The two must be used together as required by the implementation of the RTD (Real-Time Drivers) package that MBDT integrates (see section **5.3 RTD Support**). The configuration of the pins must be made according to the following information:

   For each functional group inside the *Pins Tool*, a Port container must be configured inside the Port component. Please see the screenshots below, the first one taken from the *Pins Tool*, and the second one from the Port component configuration inside the *Peripherals Tool* of a .mex file.





By inserting the **PortPin Mscr** inside the PortPin configuration, the settings inside the Port component for that particular pin will be automatically populated as configured inside the *Pins Tool*. The name of the *Pins Tool* functional group must respect the following convention: **PortContainer Name + '_MBDT'**, **'MBDT'** representing the name of the Functional Group from the *Peripherals Tool*, where all the components of the project are configured.

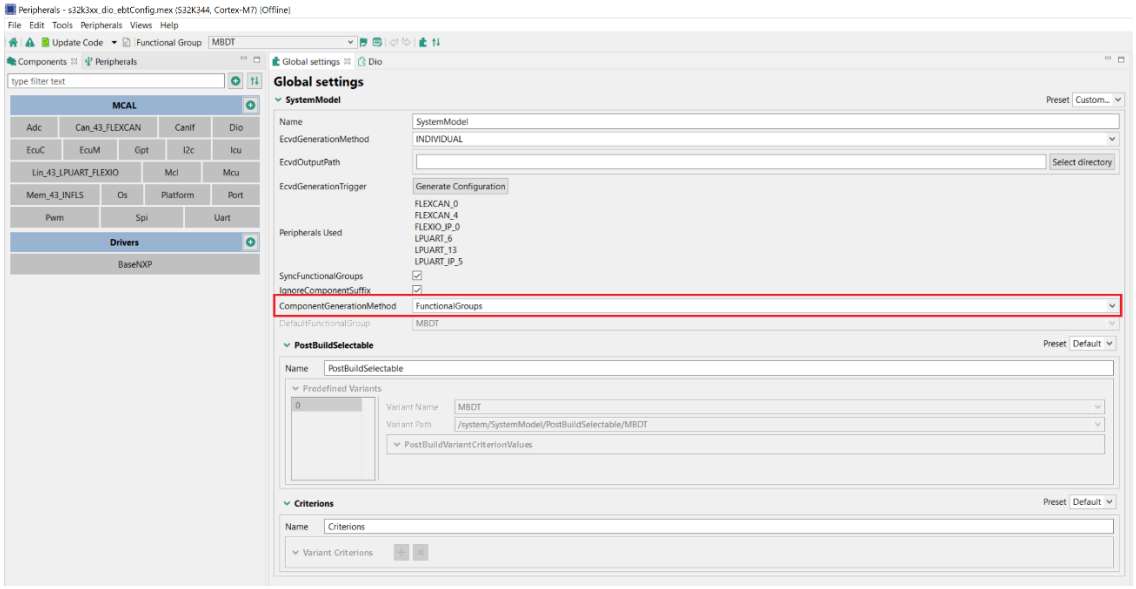
Please check one of the default configurations provided inside the toolbox for an example on how **Port** and *Pins* must be configured.

2. **Gpt** component must be enabled and configured properly if the **Gpt** option is selected as the model's Step Timer, as explained in section **3.13 Applications Timing**. For a

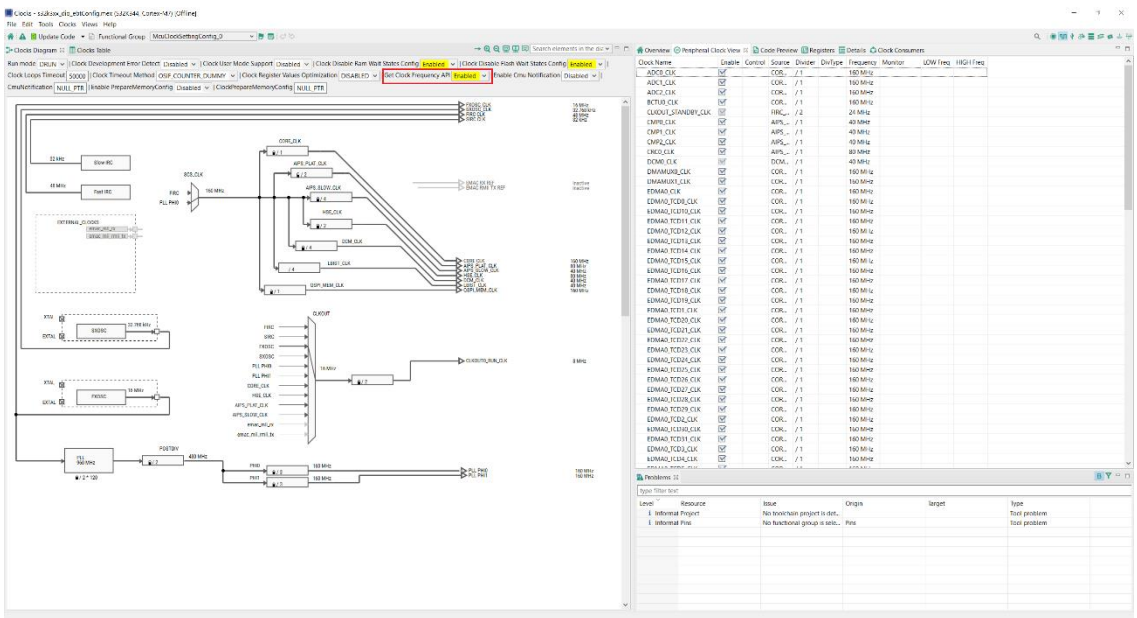
configuration example of the **Gpt** component, please check the default project provided inside the toolbox for the processor used inside your configuration (e.g S32K312-Q172.mex file if the custom project targets the S32K312 processor).

3. **Os** component needs to be enabled - no specific configuration needed - files from other components are requiring Os drivers.

4. The toolbox currently offers support only for **FunctionalGroups** as the **ComponentGenerationMethod.**

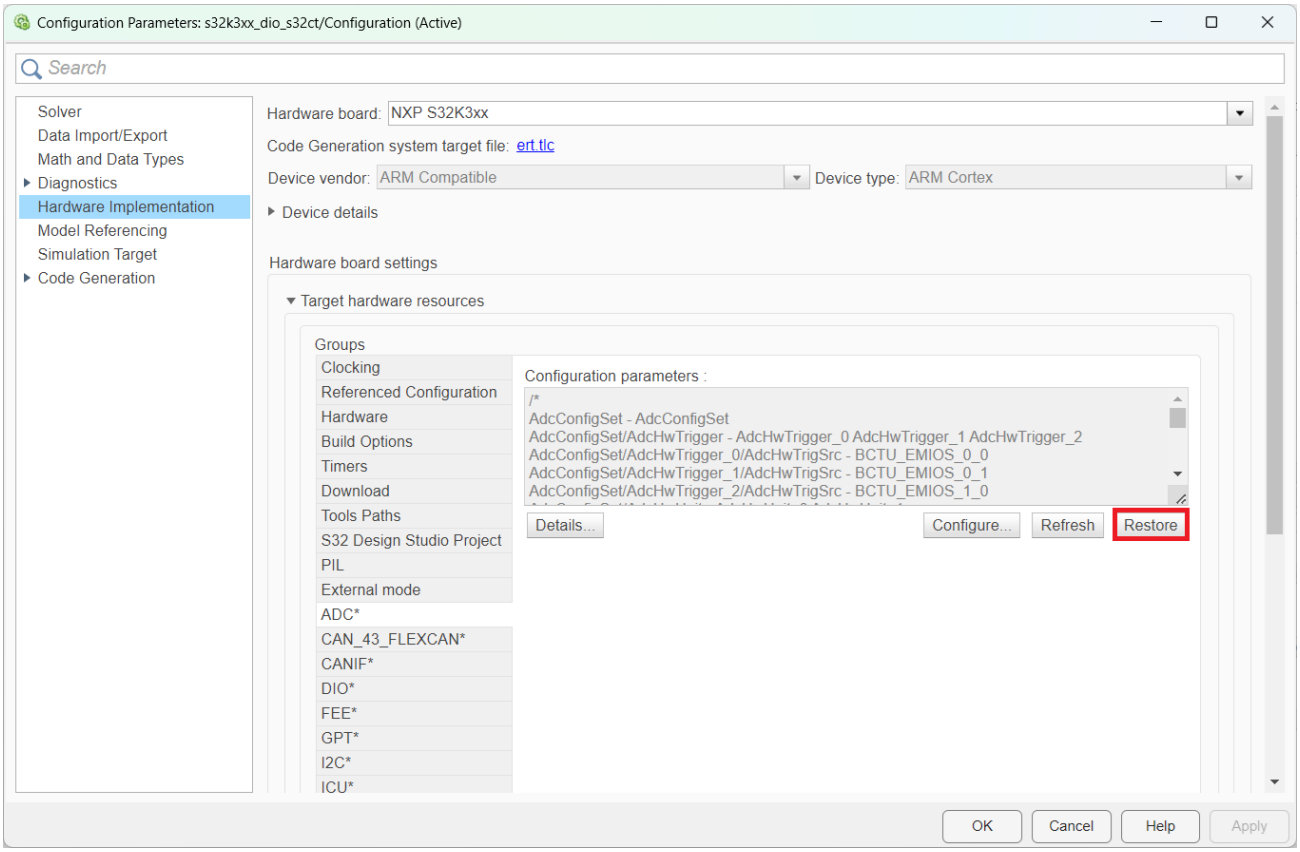


5. **Get Clock Frequency API** must be **enabled** inside the Clocks Tool for model's timing computation performed inside the toolbox.



3.7 Restore components configuration to default settings

The toolbox allows to **Restore** the configuration of a component (for models which use the EB tresos configuration tool) to the settings corresponding to the Default Configuration Template the model uses. This allows to revert modifications (if made) to the default values.



3.8 S32K3 Simulation modes

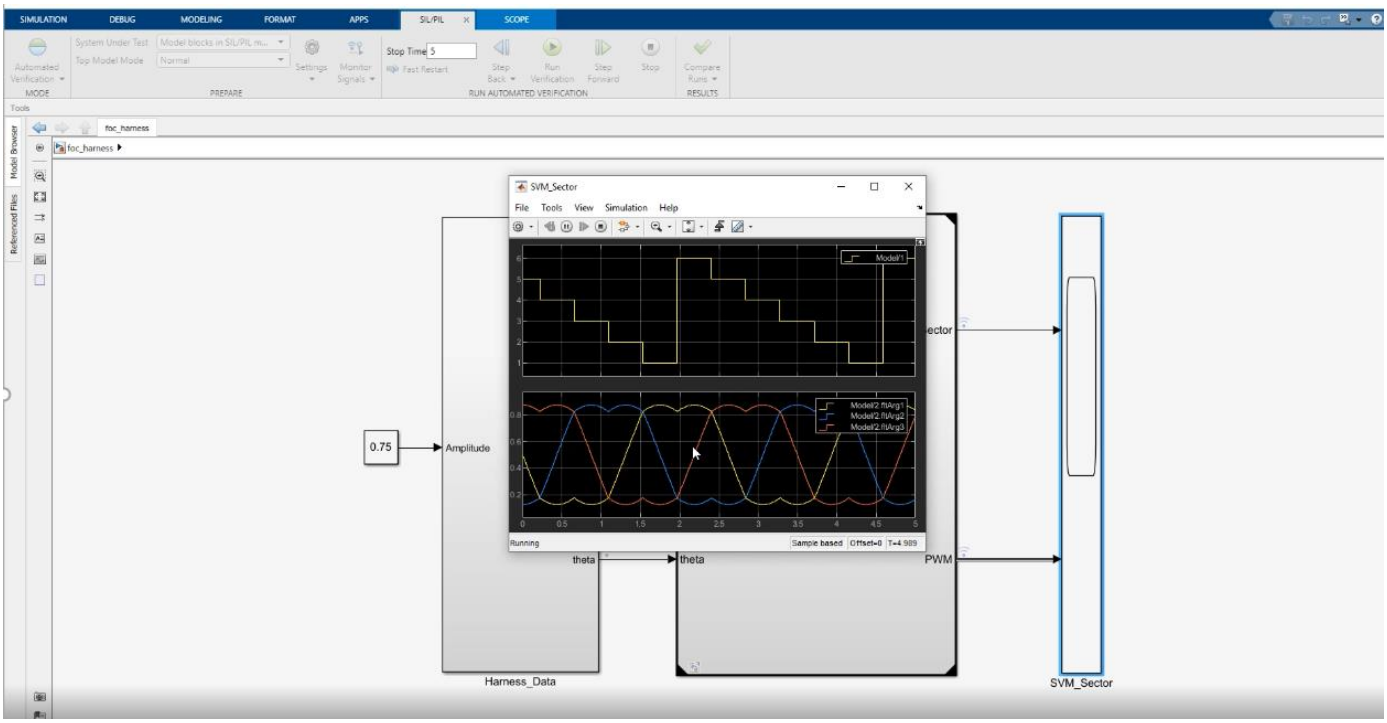
The toolbox provides support for the following Simulation modes:

- Software-in-Loop (SIL)
- Processor-in-Loop (PIL)
- External Mode

Software-in-the-Loop
A SIL simulation compiles and runs the generated code on the user's development computer. One can use such a simulation to detect early defects and fix them.

Processor-in-the-Loop
In a PIL simulation, the generated code runs on the target hardware. The results of the PIL simulation are transferred to Simulink to verify the numerical equivalence of the simulation and the code generation results. The PIL verification process is a crucial part of the design cycle to ensure that the behavior of the deployment code matches the design.

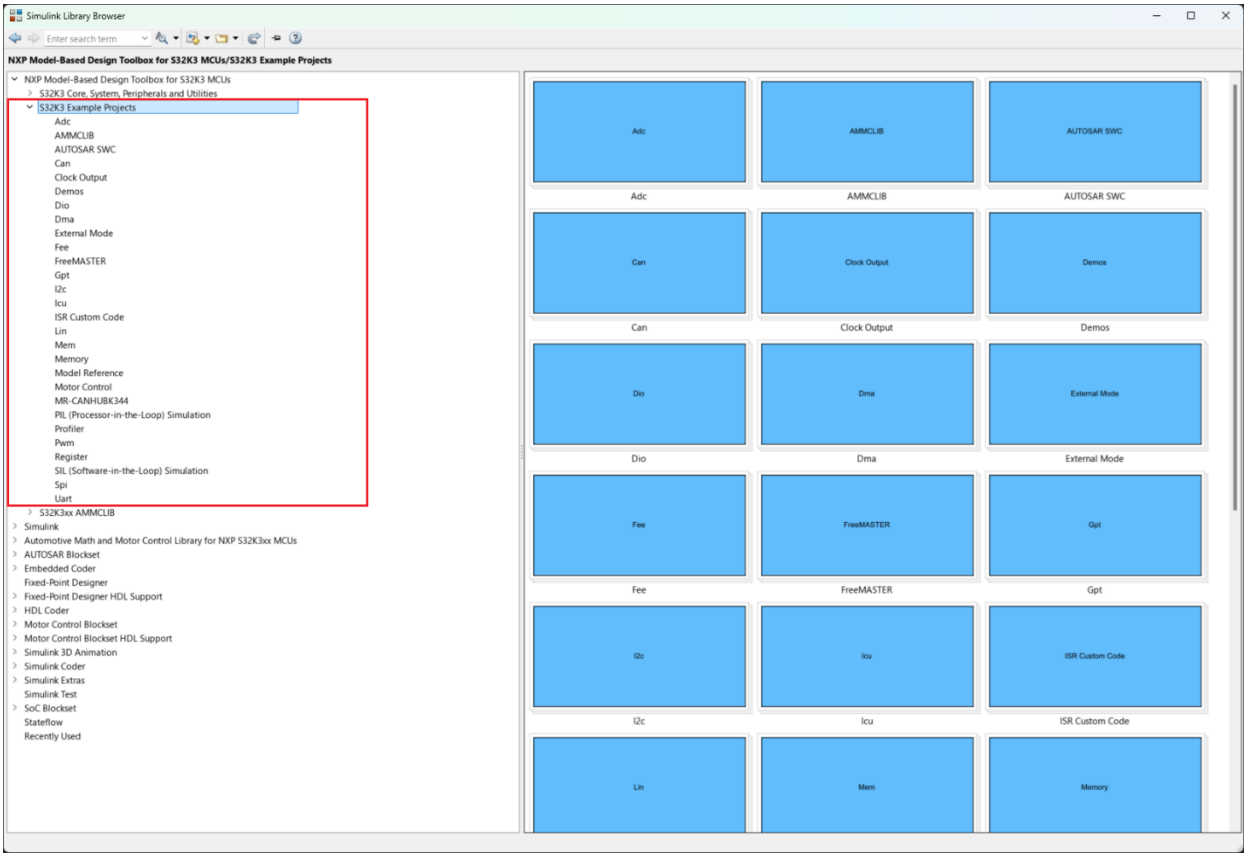
External mode
In this mode, the user can leverage Simulink to link algorithm code with the I/O driver code generated from the I/O blocks (from MATLAB). The resulting executable runs on the development computer and exchanges parameter data with Simulink via a shared memory interface.



3.9 S32K3 Example Library

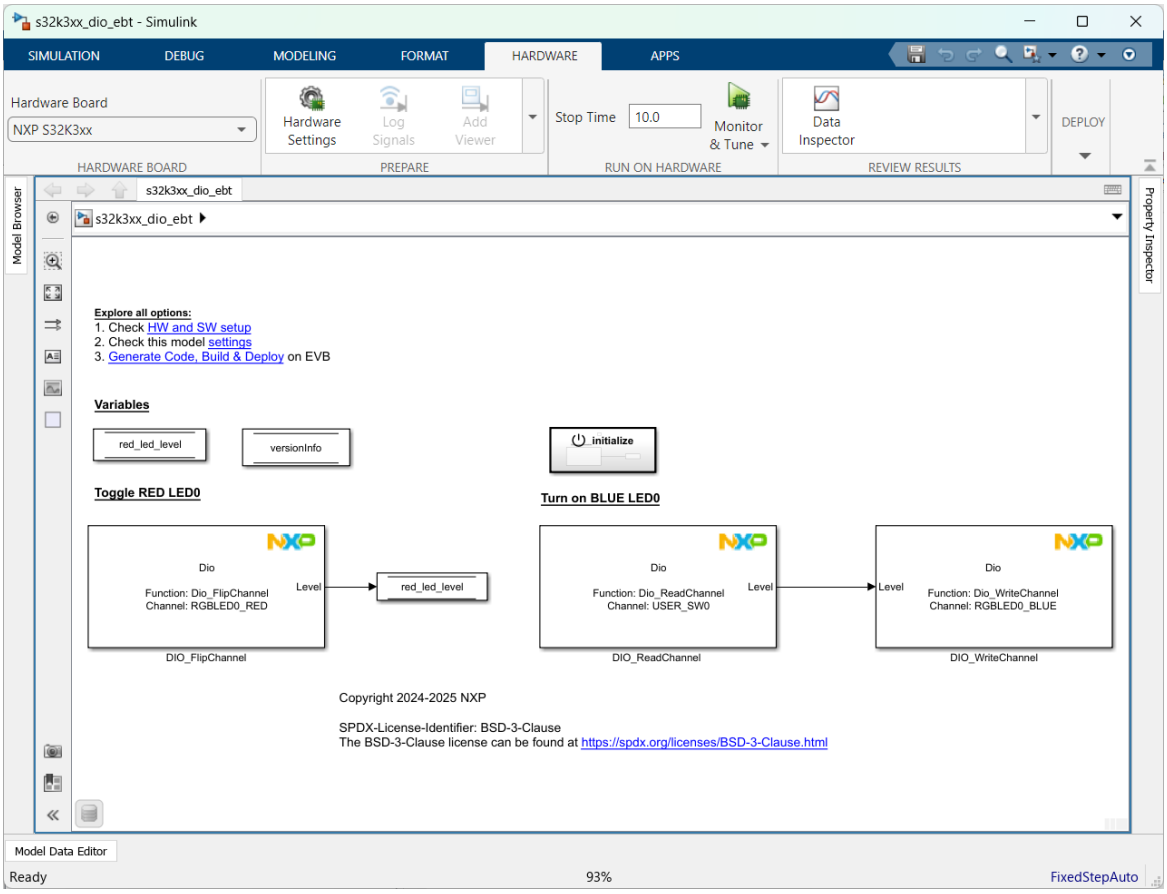
The Examples Library represents a collection of Simulink models that let you test different MCU on-chip modules and run complex applications. The toolbox provides examples targeting both S32 Configuration Tools and EB tresos, and the supported S32K3 hardware derivatives.

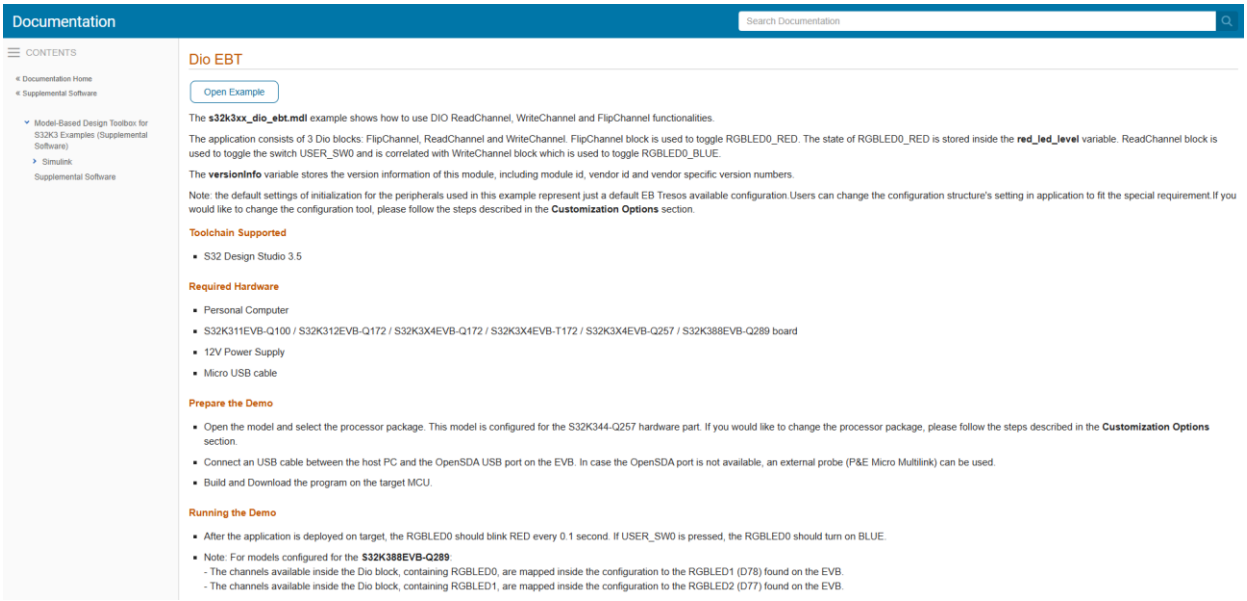
The examples are grouped in different layers that mimic a typical development flow: starting with basic building blocks that expose the MCU HW functionalities up to more complex applications that incorporate multiple building blocks.



The Simulink models shown as examples are enhanced with a comprehensive description to help users understand better the functionality that is exercised, hardware setup instructions whenever are necessary, and a result validation section.

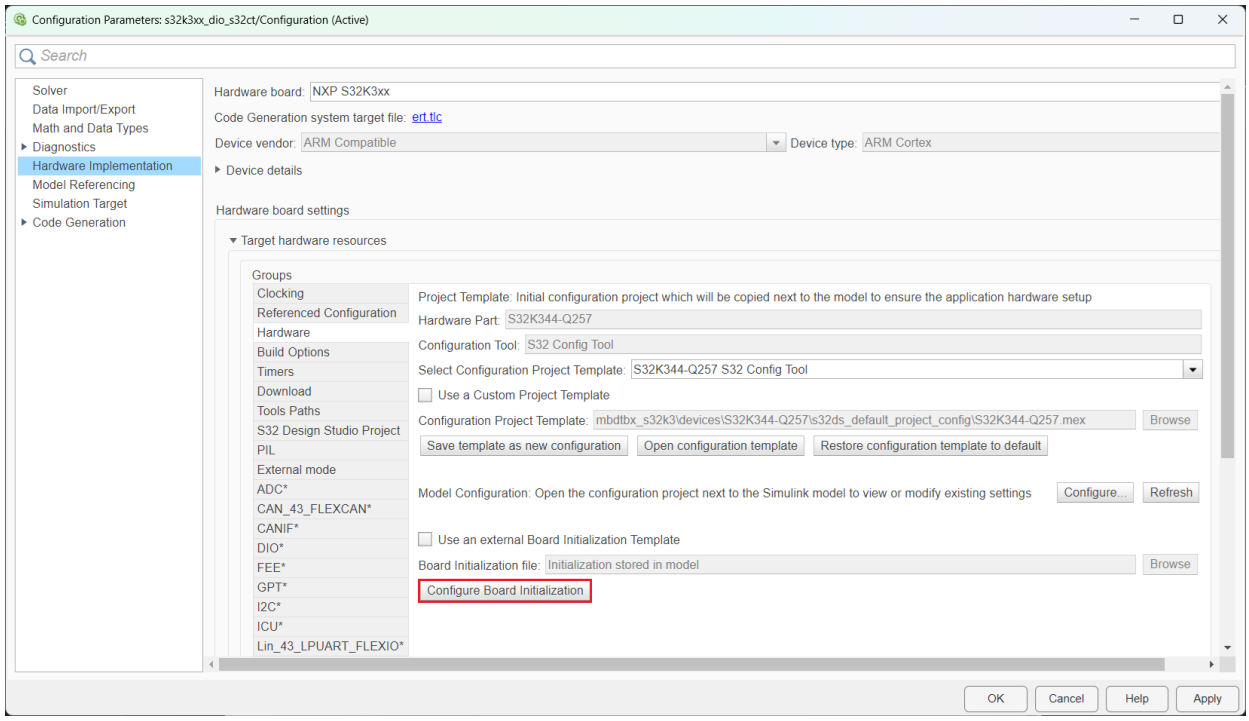
The examples are also available from the MATLAB help page.


s32k3xx_dio_ebt - Simulink
SIMULATION
DEBUG
MODELING
FORMAT
HARDWARE
APPS
Hardware Board
NXP S32K3xx
HARDWARE BOARD
Hardware Settings
Log Signals
Add Viewer
PREPARE
Stop Time
10.0
Monitor & Tune
RUN ON HARDWARE
Data Inspector
REVIEW RESULTS
DEPLOY
Model Browser
Property Inspector
s32k3xx_dio_ebt
Explore all options:
1. Check HW and SW setup
2. Check this model settings
3. Generate Code, Build & Deploy on EVB
Variables
red_led_level
versionInfo
initialize
Toggle RED LED0
Turn on BLUE LED0
Dio
Function: Dio_FlipChannel
Channel: RGBLED0_RED
Level
red_led_level
DIO_FlipChannel
Function: Dio_ReadChannel
Channel: USER_SW0
DIO_ReadChannel
Function: Dio_WriteChannel
Channel: RGBLED0_BLUE
DIO_WriteChannel
Copyright 2024-2025 NXP
SPDX-License-Identifier: BSD-3-Clause
The BSD-3-Clause license can be found at https://spdx.org/licenses/BSD-3-Clause.html
Model Data Editor
Ready
93%
FixedStepAuto

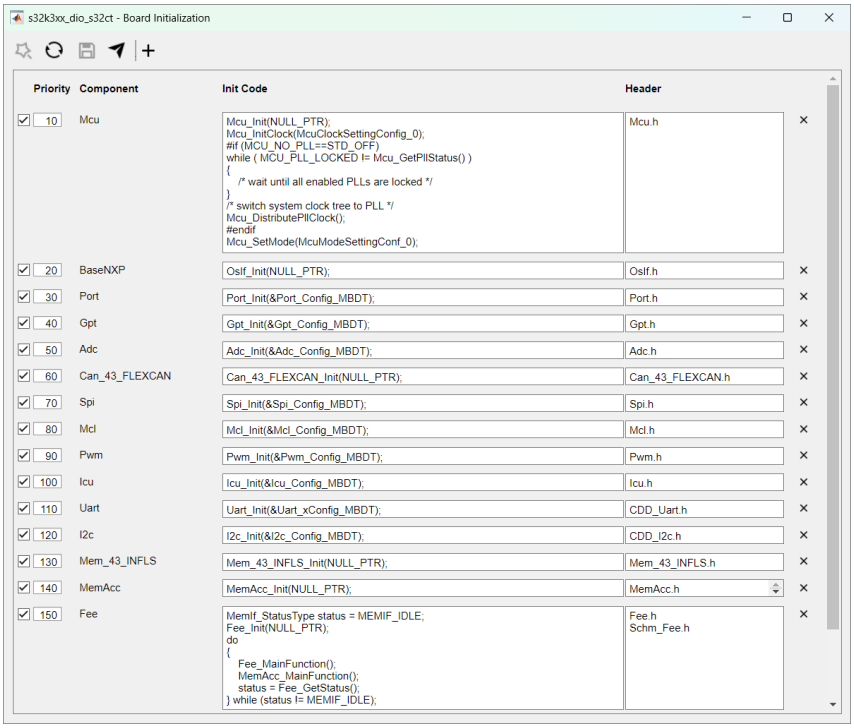

Documentation
Search Documentation
CONTENTS
Dio EBT
Open Example
The s32k3xx_dio_ebt.mdl example shows how to use DIO ReadChannel, WriteChannel and FlipChannel functionalities.
The application consists of 3 Dio blocks: FlipChannel, ReadChannel and WriteChannel. FlipChannel block is used to toggle RGBLED0_RED. The state of RGBLED0_RED is stored inside the red_led_level variable. ReadChannel block is used to toggle the switch USER_SW0 and is correlated with WriteChannel block which is used to toggle RGBLED0_BLUE.
The versionInfo variable stores the version information of this module, including module id, vendor id and vendor specific version numbers.
Note: the default settings of initialization for the peripherals used in this example represent just a default EB Tresos available configuration.Users can change the configuration structure's setting in application to fit the special requirement.If you would like to change the configuration tool, please follow the steps described in the Customization Options section.
Toolchain Supported
S32 Design Studio 3.5
Required Hardware
Personal Computer
S32K311EVB-Q100 / S32K312EVB-Q172 / S32K3X4EVB-Q172 / S32K3X4EVB-T172 / S32K3X4EVB-Q257 / S32K388EVB-Q289 board
12V Power Supply
Micro USB cable
Prepare the Demo
Open the model and select the processor package. This model is configured for the S32K344-Q257 hardware part. If you would like to change the processor package, please follow the steps described in the Customization Options section.
Connect an USB cable between the host PC and the OpenSDA USB port on the EVB. In case the OpenSDA port is not available, an external probe (P&E Micro Multilink) can be used.
Build and Download the program on the target MCU.
Running the Demo
After the application is deployed on target, the RGBLED0 should blink RED every 0.1 second. If USER_SW0 is pressed, the RGBLED0 should turn on BLUE.
Note: For models configured for the S32K388EVB-Q289:
- The channels available inside the Dio block, containing RGBLED0, are mapped inside the configuration to the RGBLED1 (D78) found on the EVB.
- The channels available inside the Dio block, containing RGBLED1, are mapped inside the configuration to the RGBLED2 (D77) found on the EVB.

3.10 Board Initialization

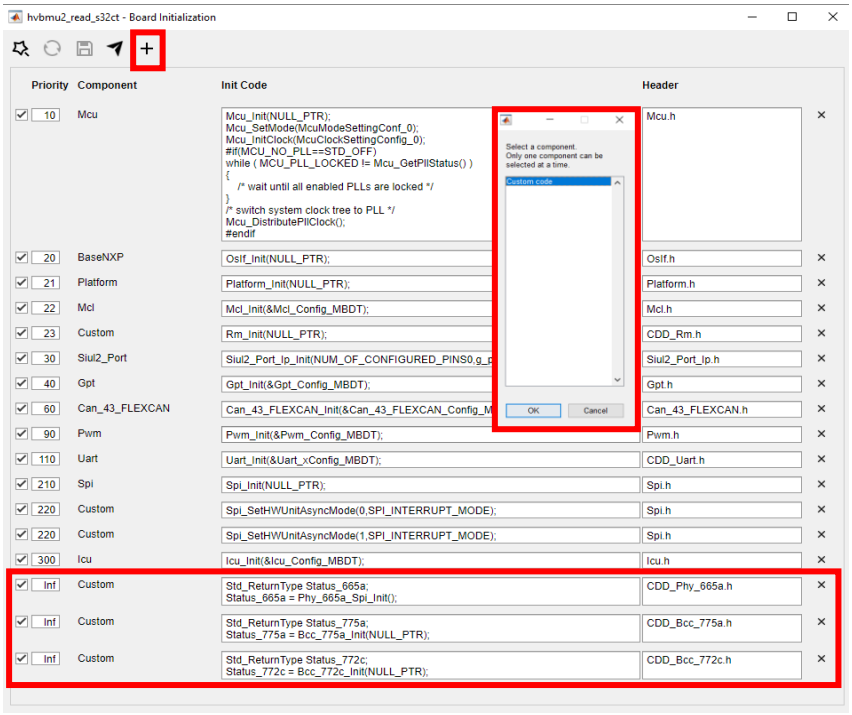
The Model-Based Design Toolbox for S32K3 generates the component's peripherals initialization function calls as configured in the **Board Initialization** window, which can be accessed from the **Hardware** group under the **Target Hardware Resources**, by pressing the **Configure Board Initialization** button.



The button will open the **Board Initialization** window, as illustrated below. The toolbox provides a default configuration including function calls for initializing the clocks, followed by pins and a custom order for the rest of the peripherals which have been configured in the external hardware project.

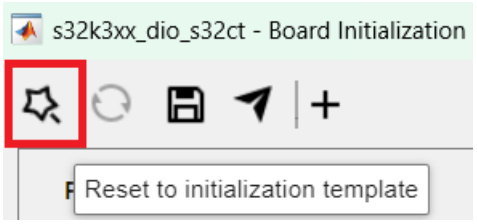


If the initialization sequence needs to be modified for a specific application, or if **Custom Code** needs to be executed during initialization, the users can enhance the default provided sequence with desired code, by pressing the **Add** button, which will provide access to defining a new entry in the Board Initialization UI.

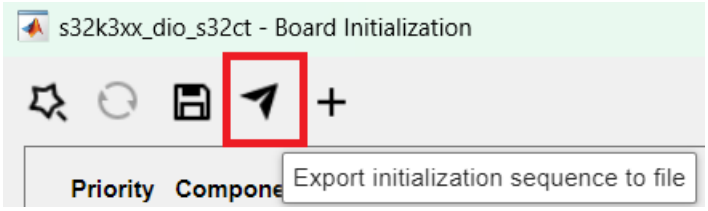


After such changes have been performed the **Save changes** button will associate the new created sequence to the model and this will be used for the process of building the application.

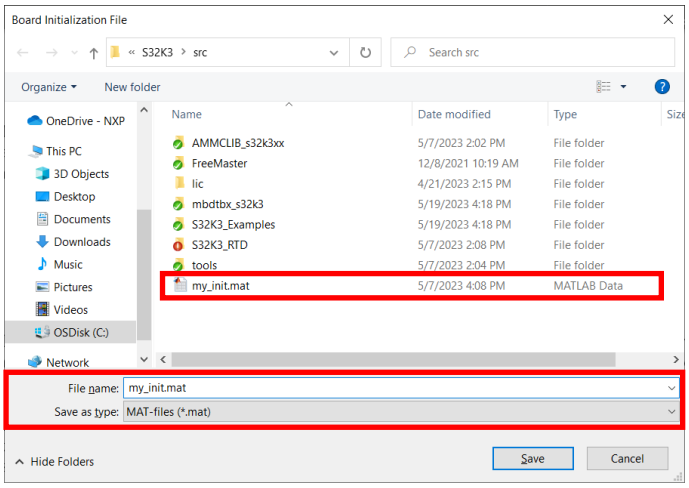
In case you would like to restore the initialization to the default one, the users can press the **Reset to initialization template** button, which becomes grayed out when the sequence used by the model corresponds to the default template.



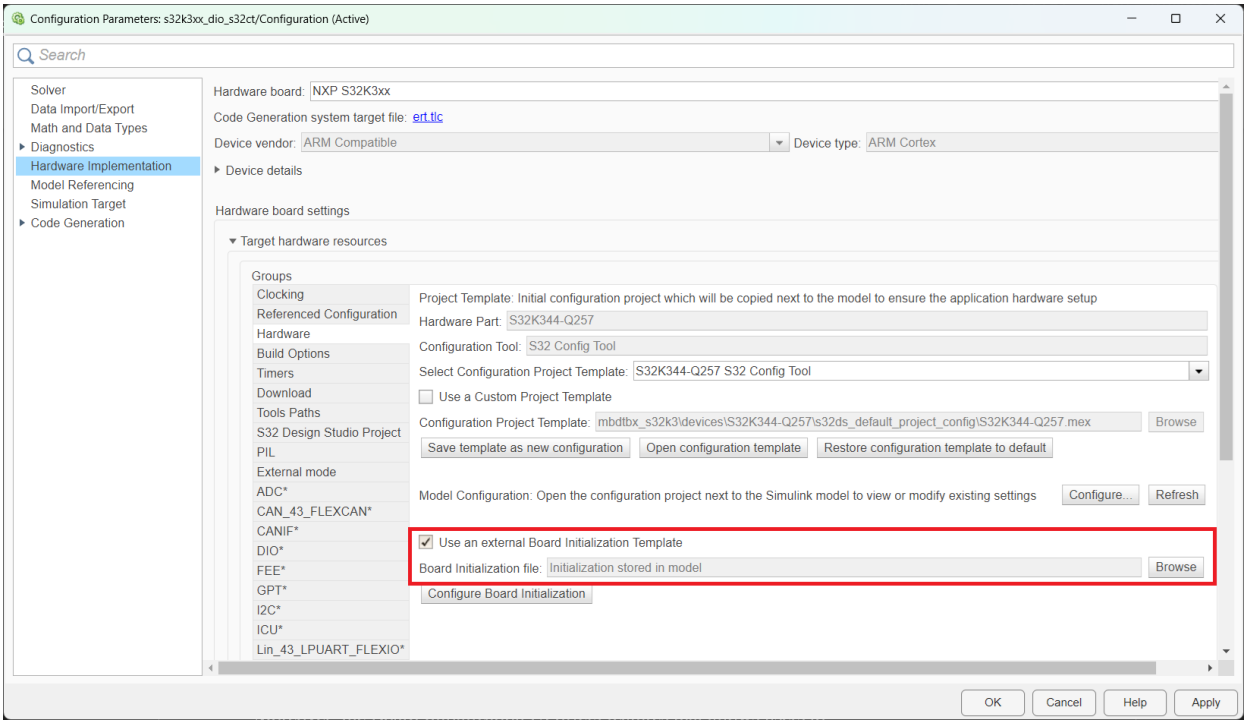
Moreover, the toolbox provides the option to export the initialization sequence to a file which can be later used for other models as well – in this way, the customization of the board initialization sequence can be done only once, even if applicable for other models as well.



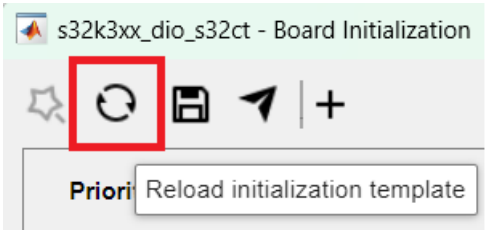
The **Export initialization sequence to file** button will prompt the Windows Explorer for the users to choose the location where the initialization will be stored. The sequence will be exported in a .mat file which can be named as desired and saved accordingly.



Such a file can be then imported as an external **Board Initialization Template** for other applications which require the same sequence.



By enabling the **Use an external Board Initialization Template** checkbox, users can **Browse** for the **.mat** file containing the sequence which needs to be used for the model. Reopning the **Board Initialization UI** or pressing the **Reload initialization template** button, the new sequence will be displayed in the UI and inspected/customized as desired.



Moreover, the Board Initialization UI offers features that allows users to:

- enable/disable the initialization of a certain component by checking/unchecking each component's checkbox;
- add/remove a component from the Board Initialization by using the **x** button;
- change the order of the initialization function calls by editing the **Priority** field;

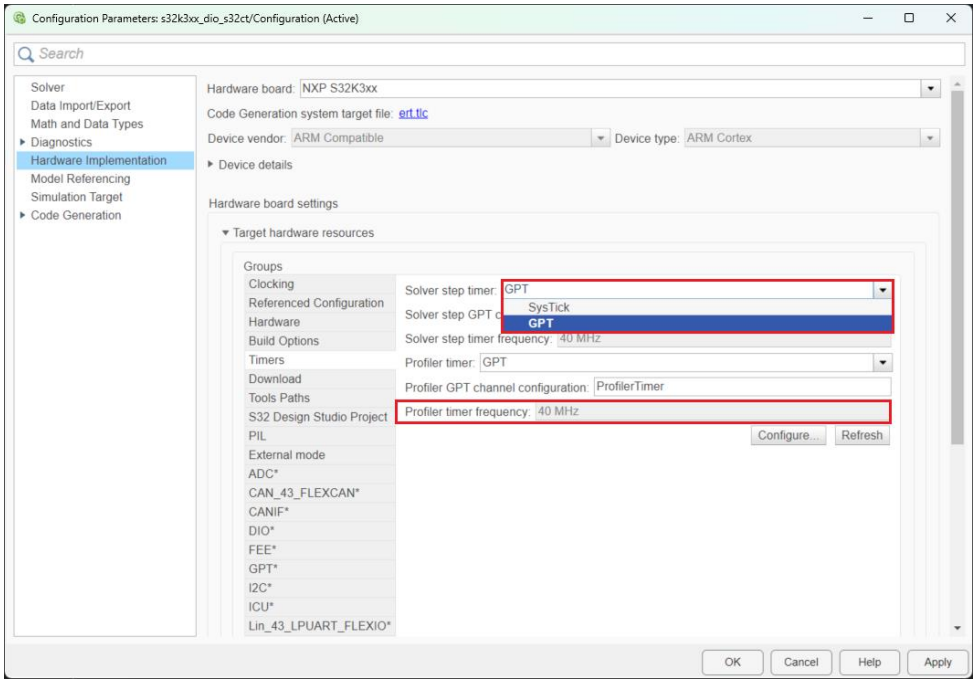
3.11 Applications Timing

The Model-Based Design Toolbox for S32K3 allows users to configure parameters related to the timing of the applications from the **Timers** group, in the model's **Configuration Parameters**. The MBDT models execute on target periodically, and this is implemented with the help of a timer. This timer is loaded with a configurable period and triggers an interrupt whenever the timeout is reached - inside its Interrupt Service Routine the application code is executed.

Users can choose the timer to be used for the model's step from the configured options available in the **Solver step timer** dropdown. Similarly, for the code execution profiling, the suitable timer can be set from the options of the **Profiler timer** dropdown.

The frequencies of the selected timers are displayed according to their settings in the configuration project associated to the model.

The **Configure** button allows the users to access directly this configuration by opening the model's associated project in the external configuration tool, while the **Refresh** button sets the new frequencies values according to the changes made in the project or in the model's **Configuration Parameters** (e.g after changing the Hardware Part, for updating the displayed frequencies of the Timers, the Refresh button should be pressed).

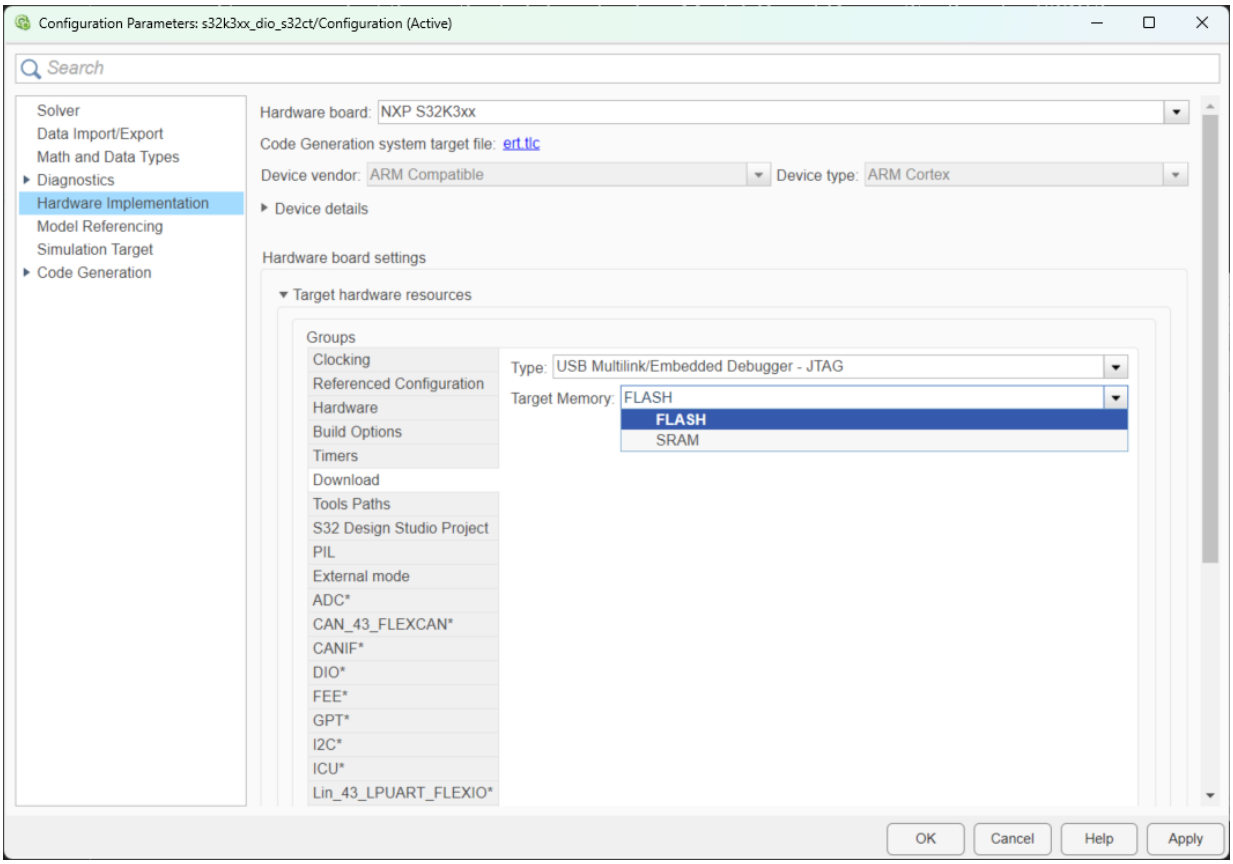


3.12 Applications download on target

Users can select the method of downloading Model-Based Design Toolbox for S32K3 applications on target from the **Download** group.

The download **Type** can be selected from the available options which support the on-board Embedded Debugger, and external probes such as **P&E Micro Multilink Debugger** and **JLink**.

The toolbox provides support for downloading applications in both FLASH or SRAM.



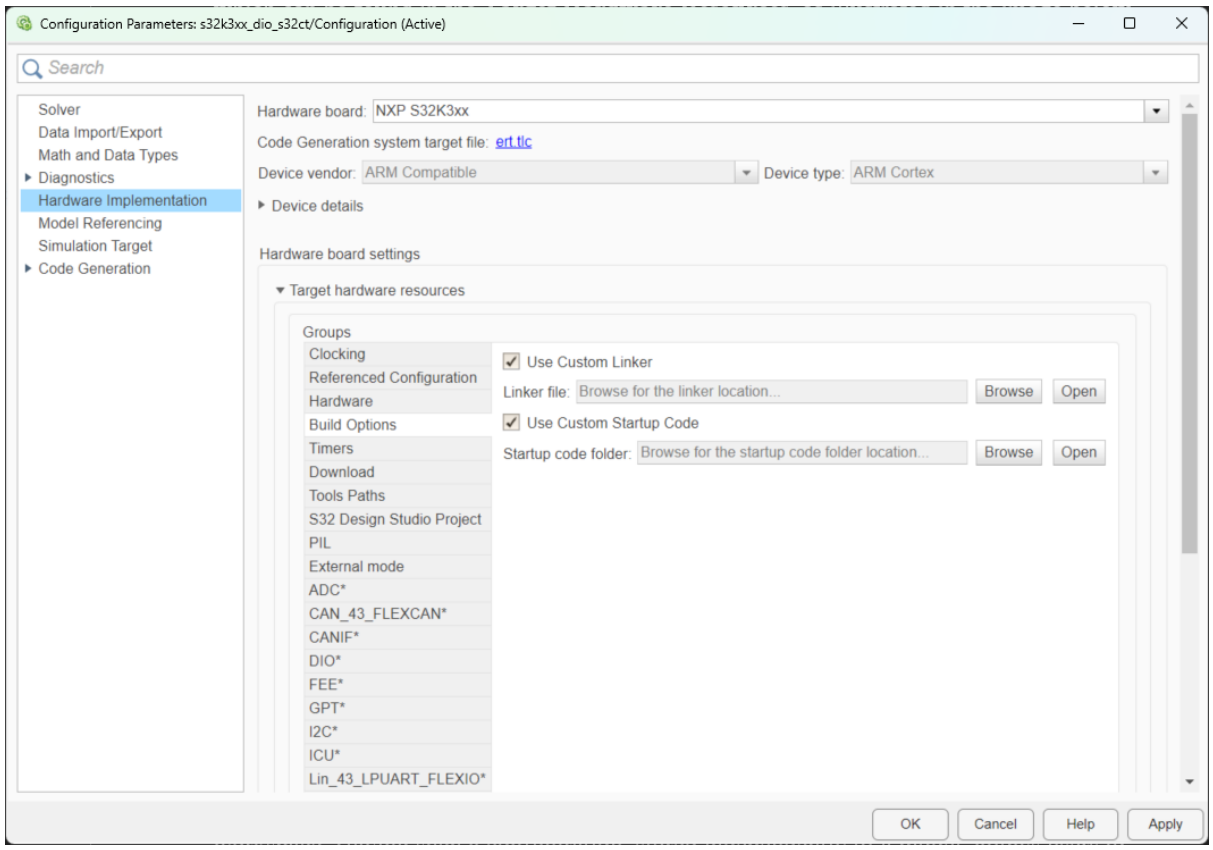
3.13 Custom Linker File and Startup Code

During the build process of Model-Based Design Toolbox for S32K3 applications, default linker files and startup code will be used, according to the **Hardware Part** and **Target Memory** selections.

However, users can choose to use custom files for this process, from the **Build Options** group which can be found in the **Target Hardware Resources**, as illustrated in the image below.

By enabling the **Use Custom Linker** or/and **Use Custom Startup Code** checkboxes, this feature is activated, allowing the users to **Browse** for specific files. In case **Custom Startup Code** needs to be used for a model, please note that all the files implementing the startup sequence must be placed in the same folder which needs to be selected when pressing the **Browse** button.

Moreover, an additional button allows Opening the selected **Linker file** or **Startup code folder** for further inspection and eventual edits.

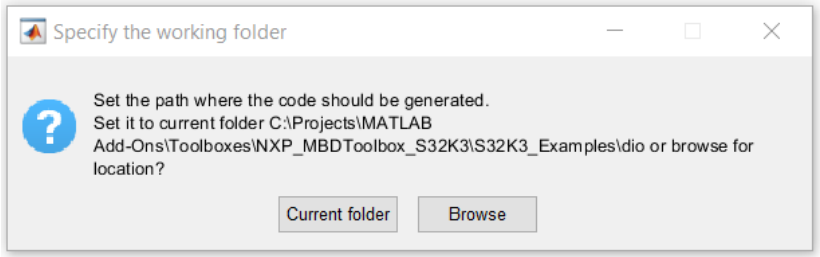


3.14 Use Referenced Configurations

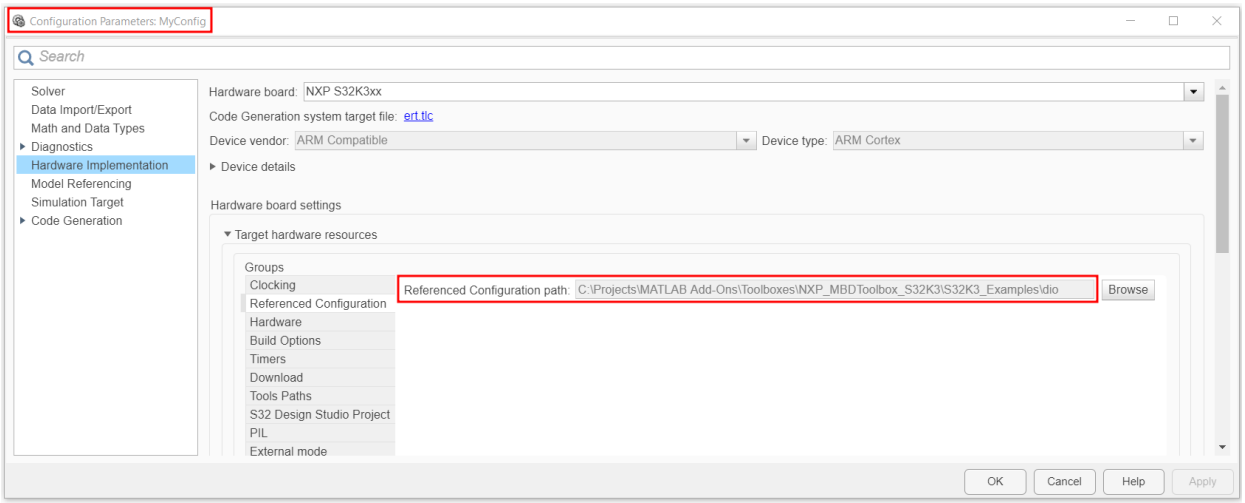
The Model-Based Design Toolbox for S32K3 enables the usage of Referenced Configurations, a Simulink feature which allows users to share the configuration of an application with multiple models.

In Simulink, new applications configurations, represented by the settings in the **Configuration Parameters** window, can be created and saved in Simulink data dictionaries or in the base workspace. Having such a configuration, stored independently of a model, allows users to access it as a template for all the developed applications that are in need of such settings. Moreover, such a configuration can be also propagated for referenced models, ensuring a faster process of setting the necessary parameters for various applications. For more details on how to create a Configuration Reference, how to use it, and how it can be propagated across multiple models, please consult the following link and the references inside it:
https://www.mathworks.com/help/simulink/ug/referencing-configuration-sets.html

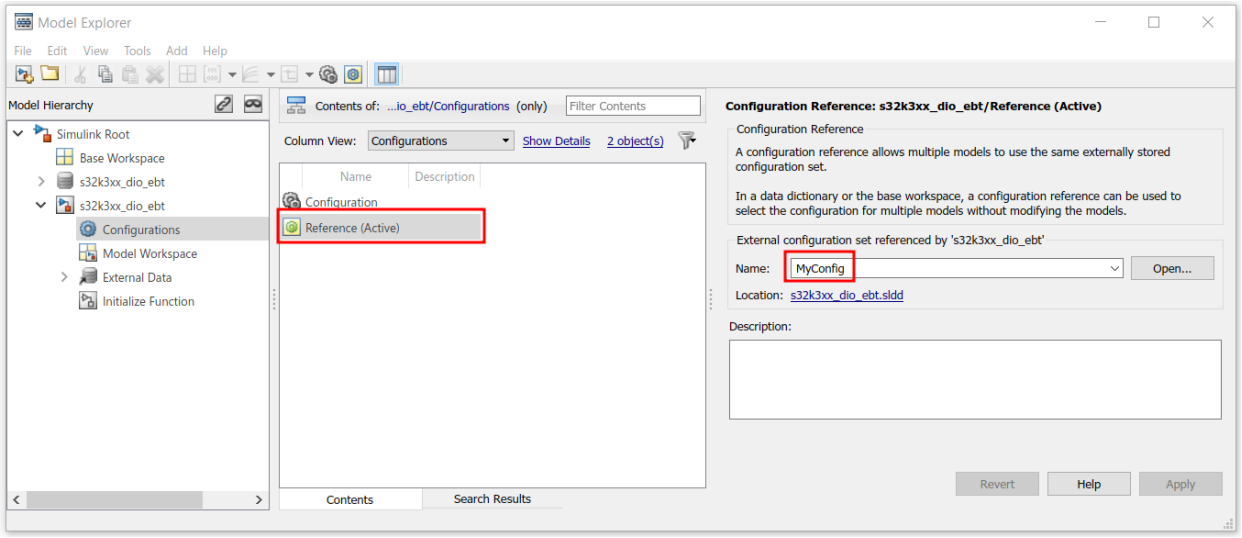
Since a configuration can be created indepentently of a specific model, when setting it for a **NXP S32K3xx** target, the user will be prompted to select the path where the hardware specific files created for the confguration will be placed (e.g. external tool configuration project - based on the selected **Hardware Part** and **Configuration Tool**).



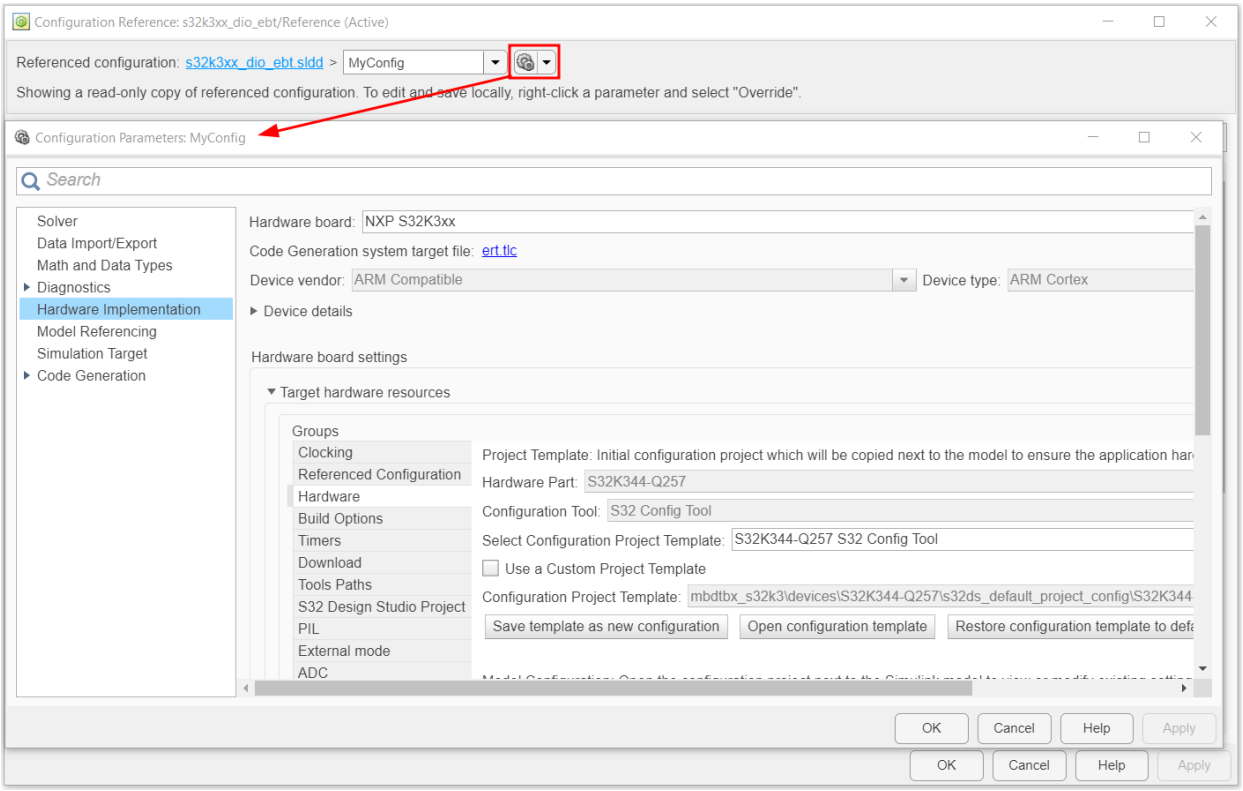
This location can be later changed, if desired, by pressing the **Browse** button from the **Referenced Configuration** group, under the **Target Hardware Resources.** Please see the image below, illustrating the settings of the configuration named **MyConfig**.



Inside the Model-Based Design Toolbox for S32K3, a model can be set to use a Reference to such a Configuration from the **Model Explorer** window, as illustrated below for the **s32k3xx_dio_ebt.mdl** application. The configuration **MyConfig** will store the application settings that this model will use.



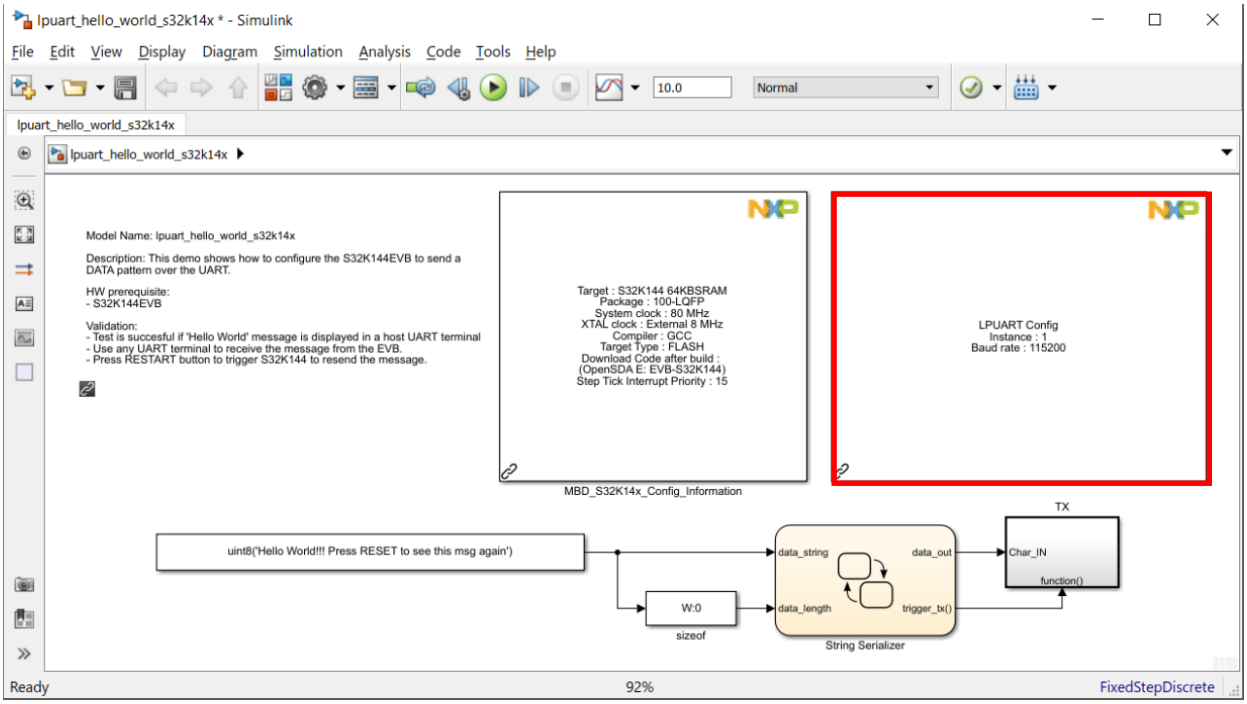
After setting **MyConfig** as a Referenced Configuration for the **s32k3xx_dio_ebt.mdl**, opening the **Hardware Settings** of the model will display the following window. Please note that the settings will be grayed out, since they will show a read-only copy of the Referenced Configuration. **MyConfig** parameters can be accessed by clicking the button highlighted in the image below.



4 Workflow and framework changes

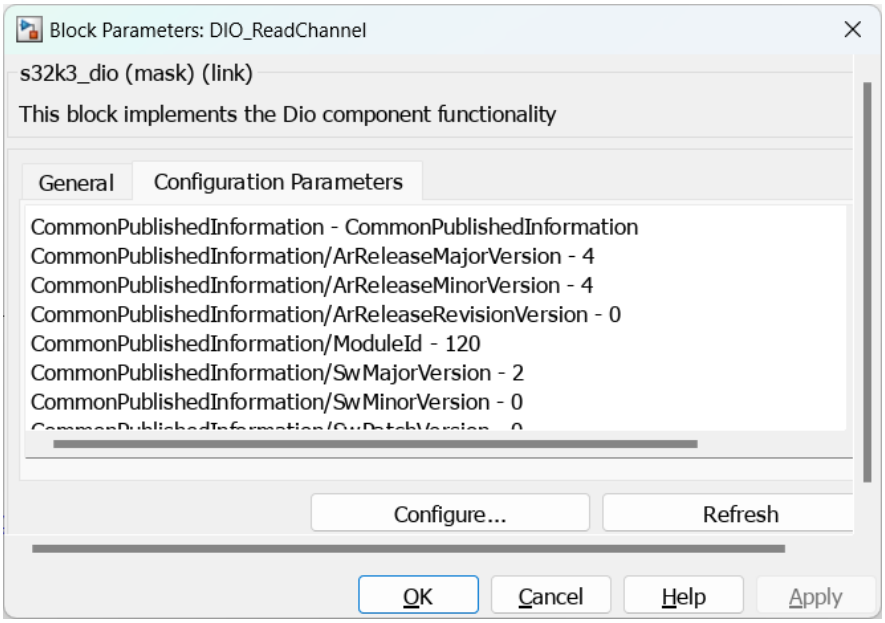
4.1 Using external configuration tools

One of the major differences between older toolboxes (e.g. for S32K1xx) and MBDT for S32K3 is the introduction of a new workflow, where the modeling and configuration are split. In other toolboxes, every peripheral had associated a Configuration Block. There, the user could set up some important parameters (e.g.: UART baud rate):



4.1.1 Modes of operation – BASIC and ADVANCED

Starting with this release, all peripheral Configuration Blocks are no longer needed. Instead, we provide a default configuration for all the peripherals we support. Users can see the default parameters for every such block, going in the mask, searching for the 'Configuration Parameters' tab. This use case is what we call the BASIC mode of operation – where there is no additional configuration to be made (everything is based on the default one), so no interaction with an external configuration tool is needed.



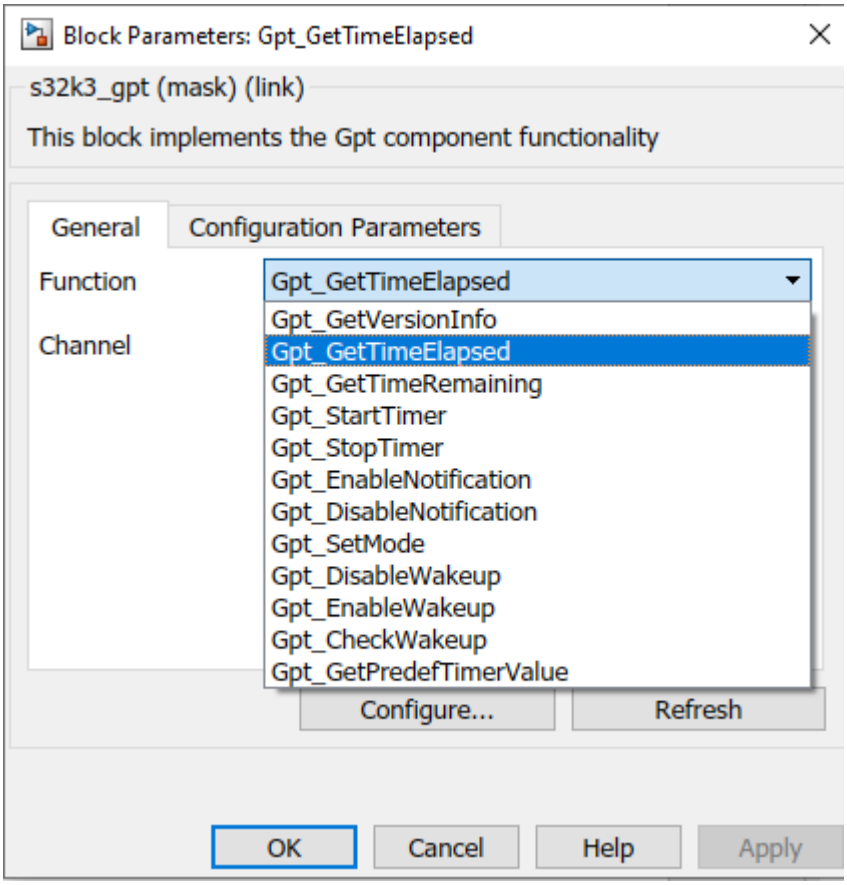
If something needs to be edited/added/removed, users can open an external configuration tool of choice (NXP S32 Configuration Tools or EB tresos). There, all the parameters of peripherals, pins and clock are available. The Simulink model is linked to the external configuration tools and is aware of changes made, so simply building the model one more time/clicking on the 'Refresh' button of the block, after the changes were saved in the external tool, fetches the new values of the parameters. This is the ADVANCED mode, where changes to the default configuration need to be made. Note that NXP S32 Configuration Tools is shipped with our toolbox, so no extra installation process needed; EB tresos needs to be installed and licensed (free of charge), following the steps in the MBDT Quick Start Guide.

4.1.2 Advantages for using external configuration tools

There are a couple of key advantages to using an external configuration tool instead of the peripheral configuration blocks. Firstly, using these dedicated tools, the user has access to all parameters for the peripherals. Additionally, full access to pins and clock configuration is provided (while the older framework has limited support for this). Secondly, splitting the modeling (algorithms, state machines, etc) from the configuration means they can be done in parallel. This can be an advantage when applications are developed in larger teams (one engineer might work on the configuration for all pins, clock, peripherals while another engineer can focus on modeling a complex algorithm in Simulink). Additionally, different configuration setups can be easily switched, without having to change anything inside the Simulink model.

4.2 Code generation based on RTD support (MCAL drivers)

In this release, the code generation is based on the Real Time Drivers. The peripheral support is different from other releases (e.g. for S32K1xx), not having a *per-peripheral* strategy, but a *per MCAL component/driver* one. For example, LPIT (low power interrupt timer) and STM (system timer module) support will not be done via separate blocks, but within one GPT (general purpose timer) block. The structure will be similar for all such blocks: a list of supported functions (MCAL standard functions and additional functions, added for optimization purposes) and a few parameters (depending on selected functionality).



5 Prerequisites

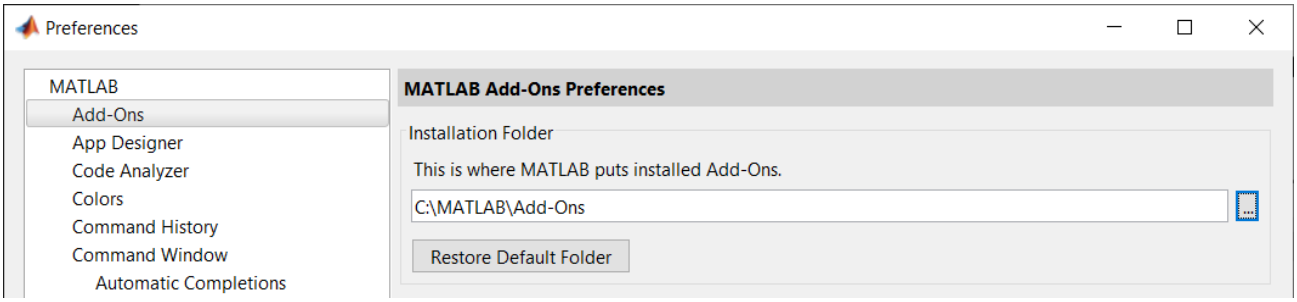
5.1 MATLAB Releases and OSes Supported

This toolbox is developed and tested to supports the following MATLAB releases:

- R2021a;
- R2021b;
- R2022a;
- R2022b;
- R2023a;
- R2023b;
- R2024a;
- R2024b;

Important: *It is **recommended** to install the* NXP's Model-Based Design Toolbox for S32K3 Series version 1.6.0 *into a location that **does not contain special characters, empty spaces, or mapped drives.***

The default location can be changed before installation by changing the Add-Ons path from MATLAB Preferences.



For a flowless development experience the minimum recommended PC platform is:

- *Windows® OS*: any x64 processor
- At least 4 GB of RAM
- At least 6 GB of free disk space.
- Internet connectivity for web downloads.

Operating System Supported

	SP Level	64-bit
Windows 7	SP1	X
Windows 10		X
Windows 11		X

5.2 Build Toolchain Support

The following compilers are supported:

Compiler Supported	Release Version
GCC for ARM Embedded Processors	V10.2.0

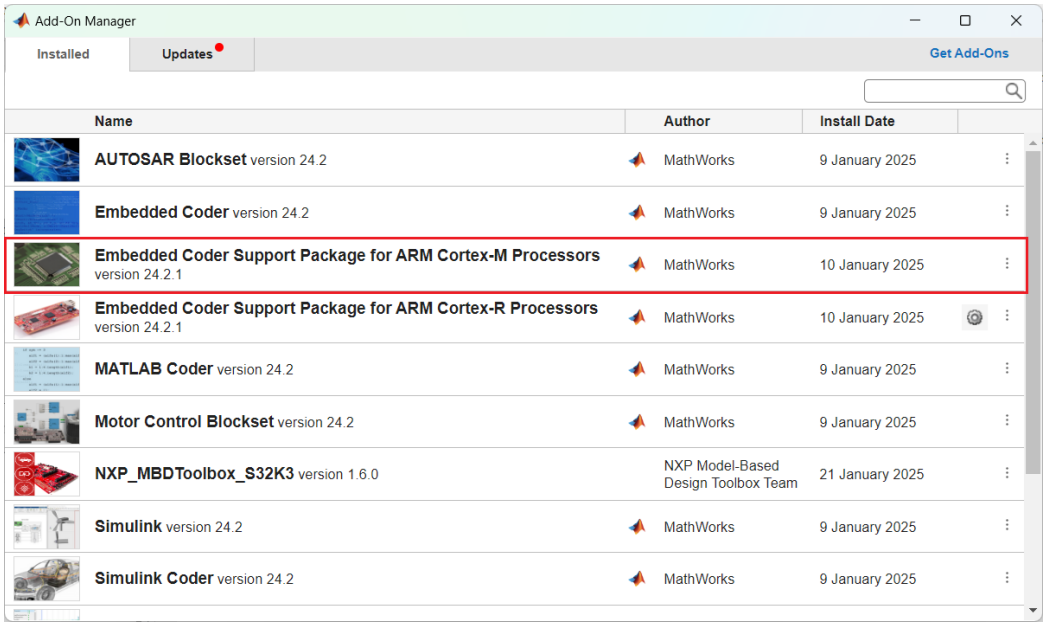
The target compiler for the Model-Based Design Toolbox needs to be configured.

The Model-Based Design Toolbox uses the Toolchain mechanism exposed by the Simulink to enable automatic code generation with Embedded and Simulink Coder toolbox. By default, the toolchain is configured for the MATLAB 2021a, R2021b, R2022a, R2022b, R2023a, R2023b, R2024a, and R2024b releases. For any other MATLAB release, the user needs to execute a toolbox m-script to generate the appropriate settings for his/her installation environment.

This is done by changing the MATLAB Current Directory to the toolbox installation directory (e.g.: `..\MATLAB\Add-Ons\Toolboxes\NXP_MBDToolbox_S32K3\`) and running the "`mbd_s32k3_path.m`" script.

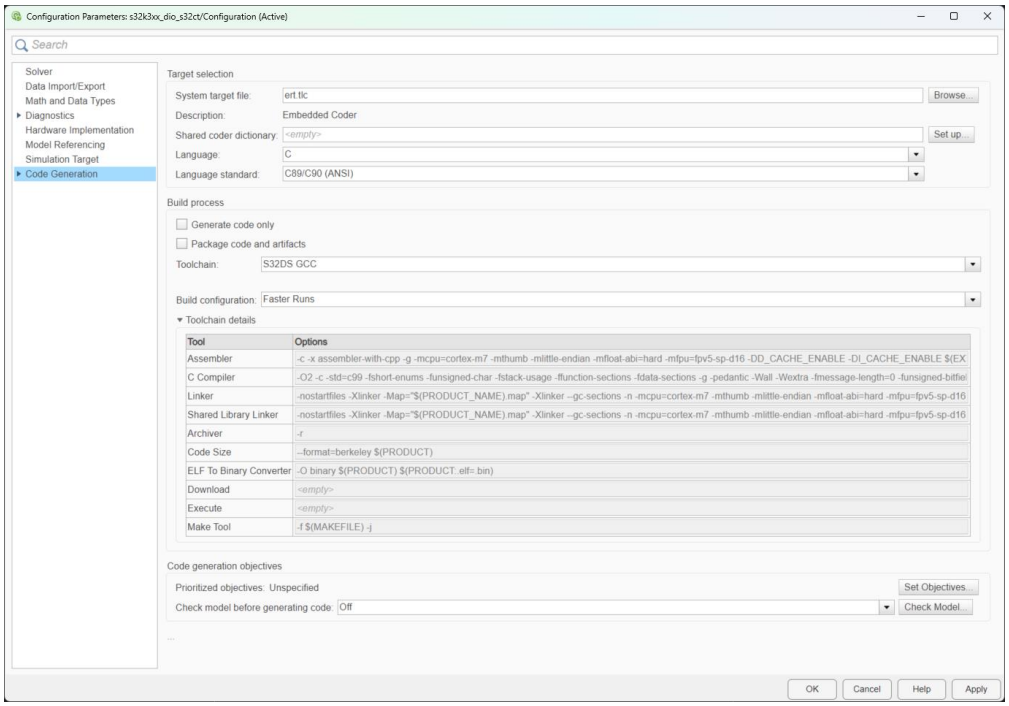
```
>> mbd_s32k3_path
Treating 'C:\MATLAB\Add-Ons\Toolboxes\NXP_MBDToolbox_S32K3 as MBD
Toolbox installation root.
MBD Toolbox path prepended.
Creating folders for the target 'NXP S32K3xx' in the folder
'C:\[…]\S32K3\src\mbdtbx_s32k3\codertarget\2021a'...
Creating the framework for the target 'NXP S32K3xx'...
Registering the target 'NXP S32K3xx'...
Done.
Successful.
```

This mechanism requires users to install the Embedded Coder Support Package for ARM Cortex-M Processor as a prerequisite.



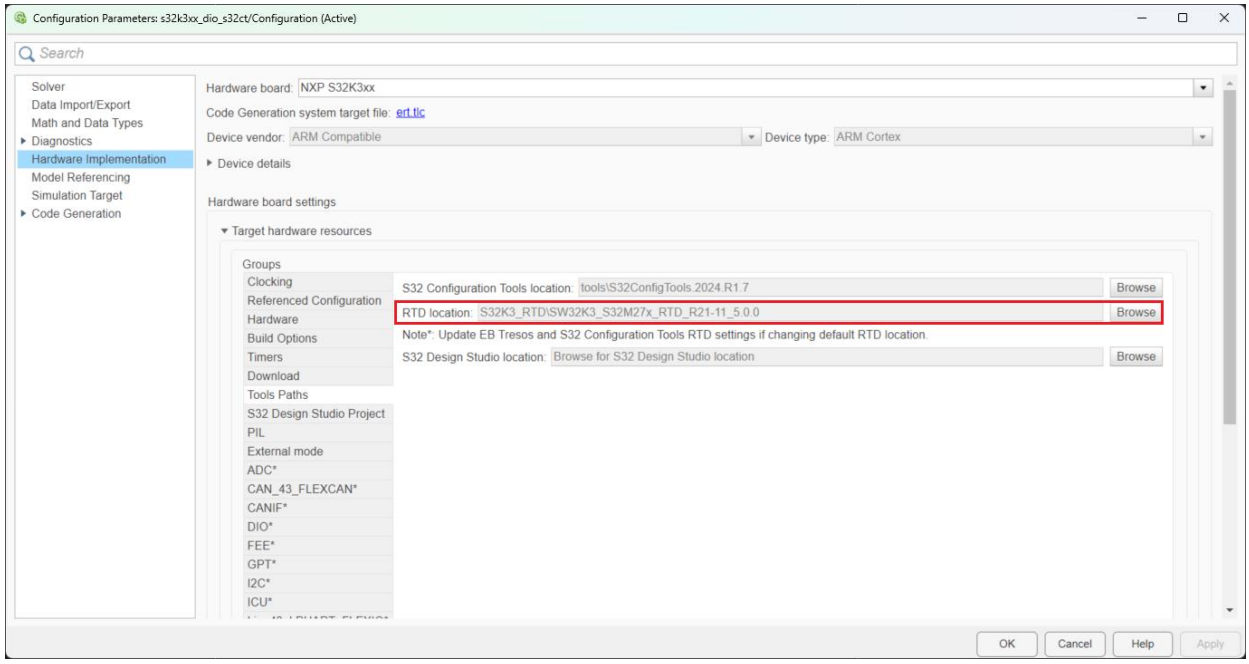
The "`mbd_s32k3_path.m`" script verifies the user setup dependencies and will issue instructions for a successful installation and configuration of the toolbox.

The toolchain can be further enhanced using the Simulink **Model Configuration Parameters** menu:



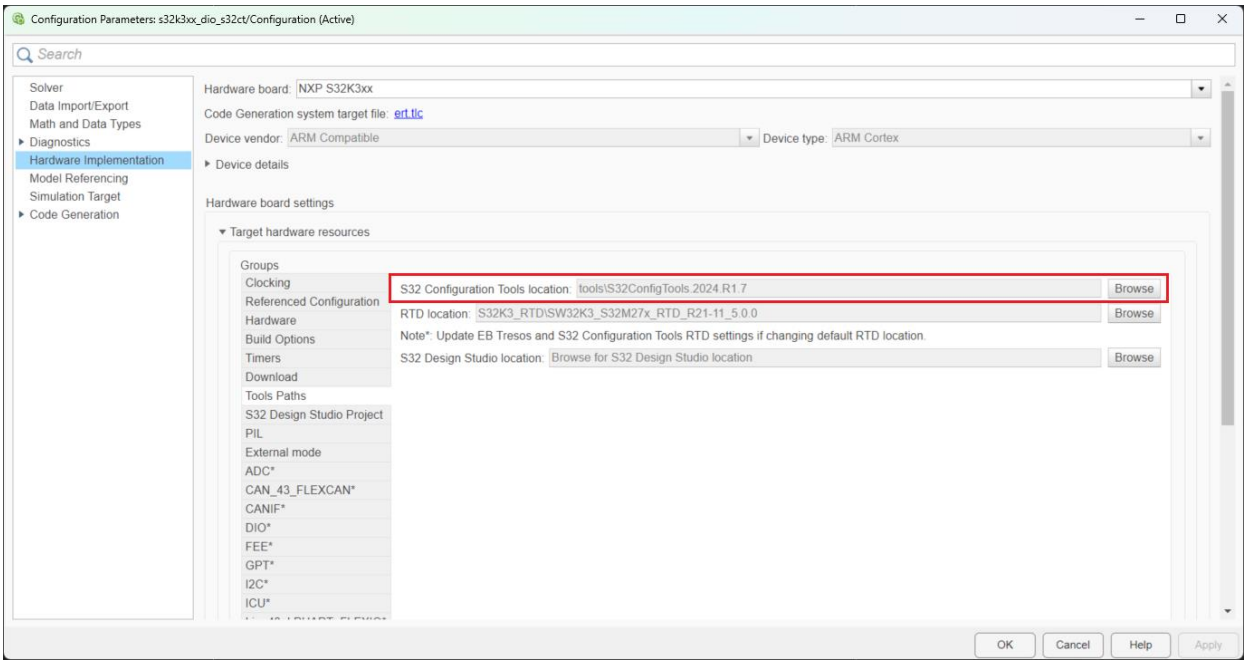
5.2 RTD Support

The toolbox is delivered with support for RTD 5.0.0. The user can change the RTD package from the Simulink **Model Configuration Parameters** menu.



5.3 S32 Configuration Tool Support

The toolbox is delivered with Configuration Tools v2021.R1.7. The user can change the Configuration Tools version used by the Simulink from the Simulink **Model Configuration Parameters** menu.



6 Known Limitations

The list of know limitations can be found the readme.txt file that is delivered with the toolbox and can be consulted in the MATLAB Add-on installation folder of the Model-Based Design Toolbox for S32K3 Series.

7 Support Information

For technical support please sign on to the following NXP's Model-Based Design Toolbox Community: https://community.nxp.com/t5/NXP-Model-Based-Design-Tools/bd-p/mbdt

How to Reach Us:

Home Page:
www.nxp.com

Web Support:
www.nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP Semiconductor products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits or integrated circuits based on the information in this document.

NXP Semiconductor reserves the right to make changes without further notice to any products herein. NXP Semiconductor makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale Semiconductor assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. “Typical” parameters that may be provided in NXP Semiconductor data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including “Typicals”, must be validated for each customer application by customer’s technical experts. NXP Semiconductor does not convey any license under its patent rights nor the rights of others. NXP Semiconductor products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the NXP Semiconductor product could create a situation where personal injury or death may occur. Should Buyer purchase or use NXP Semiconductor products for any such unintended or unauthorized application, Buyer shall indemnify and hold NXP Semiconductor and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that NXP Semiconductor was negligent regarding the design or manufacture of the part.

MATLAB, Simulink, Stateflow, Handle Graphics, and Real-Time Workshop are registered trademarks, and TargetBox is a trademark of The MathWorks, Inc.
Microsoft and .NET Framework are trademarks of Microsoft Corporation.
Flexera Software, Flexlm, and FlexNet Publisher are registered trademarks or trademarks of Flexera Software, Inc. and/or InstallShield Co. Inc. in the United States of America and/or other countries.
NXP, the NXP logo, CodeWarrior and ColdFire are trademarks of NXP Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Flexis and Processor Expert are trademarks of NXP Semiconductor, Inc. All other product or service names are the property of their respective owners